

MODULE 4

The MongoDB Query Language

Read, filter, shape, insert, update, upsert and delete training_store documents — and prove every write did what you meant.

Read and filter

Query operators

**Nested fields and
arrays**

**Insert, update,
upsert**

Delete and verify

LEARNING OBJECTIVES

What you will be able to do by the end of this module.

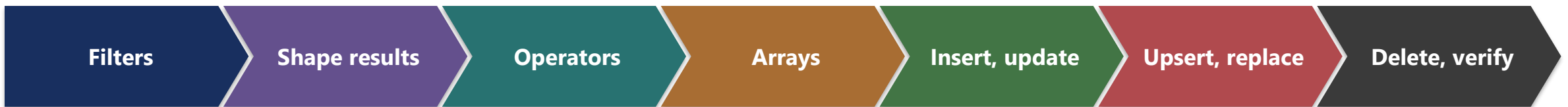


BY THE END OF THIS MODULE, YOU CAN

1. Read documents with find() and findOne()
2. **Shape results:** project, sort, page, count
3. Combine comparison and logical operators
4. Query nested fields and arrays
5. Insert, update, upsert and replace
6. Delete safely and read write results



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHICH

documents does my filter match?

WHAT

fields should come back?

HOW

do operators combine?

WHEN

is \$elemMatch required?

HOW

do I change only what I mean?

HOW

do I prove a write was right?

KEY TAKEAWAY A good query starts from the real field names, values and BSON types in the documents.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 Query filter

A document that says which documents match, e.g. { category: "BOOK" }. {} matches every document.

3 Cursor

What find() returns: a pointer that hands back matching documents in batches, not all at once.

5 \$elemMatch

Requires one array element to meet every condition — not different elements for different conditions.

7 Upsert

{ upsert: true }: update the match, or insert a new document when nothing matches the filter.

2 Projection

The second find() argument: it chooses which fields come back, e.g. { name: 1, _id: 0 }.

4 Dot notation

A quoted path into nested fields and arrays, such as "attributes.sizes" or "items.sku".

6 Update operator

\$set, \$inc, \$push and friends: change named fields in place instead of replacing the document.

8 Write result

What every write returns: acknowledged, matchedCount, modifiedCount, insertedId, deletedCount.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

BSON

Binary JSON

JSON

JavaScript Object Notation

USB

Universal Serial Bus

CRUD

Create, Read, Update, Delete

MQL

MongoDB Query Language

UTC

Coordinated Universal Time

ID

Identifier

SKU

Stock Keeping Unit

ISO

International Organization for
Standardization (ISODate uses the
ISO 8601 date format)

SQL

Structured Query Language

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 3 TO MODULE 4

Module 3 designed the documents. Now we read and change them.

MODULE 3 GAVE US

- ✓ Documents shaped by access patterns
- ✓ Items embedded, customers referenced
- ✓ Decimal128 money, Date timestamps
- ✓ Four collections in training_store

NOW Query and change that model with filters shaped like the documents.



THE PATH THROUGH MODULE 4

- 1 Read: find, filters, projection, sort
- 2 Comparison, logical and element operators
- 3 Nested fields and arrays
- 4 Insert, update, upsert and replace
- 5 Delete, bulk write and verify

Part 1

Part 2

Part 3

Part 4

Part 5

KEY TAKEAWAY The document design from Module 3 decides how every query in Module 4 is written.

THE TRAINING_STORE DATA WE QUERY

Counts, field names and the traps planted for this module.



FOUR COLLECTIONS ON A FRESH LOAD

customers

6 documents

products

13 documents

orders

17 documents

reviews

6 documents



FIELD NAMES TO QUERY — EXACTLY AS STORED

products	sku, name, category, price, tags, attributes, active
customers	customerNumber, name.first, contact.email, addresses
orders	orderNumber, customerId, items, paymentStatus, fulfillmentStatus, total, createdAt
reviews	sku, productId, customerId, rating, title, body

VALUES ARE UPPERCASE category "LAPTOP", status "ACTIVE", paymentStatus "PAID" — and there is no order field called status.

KEY TAKEAWAY Look at the real field names, values and types before writing a filter — MongoDB matches exactly what is stored.

PLANTED ON PURPOSE

- ! XBAD: price is the string "49.99"
- ! C204: phone is null; C515 has none
- ! L190: inactive, tagged discontinued
- ! O5001: L100 × 1 and A410 × 2
- ! No stock field on any product

RELOAD Run load.js again before each lab so counts start fresh.

PART 1: READING DOCUMENTS

find(), findOne() and the shape of a read query.



SO FAR

- ✓ You know the module goals and key terms
- ✓ You know the fields and planted traps

NEXT QUESTION

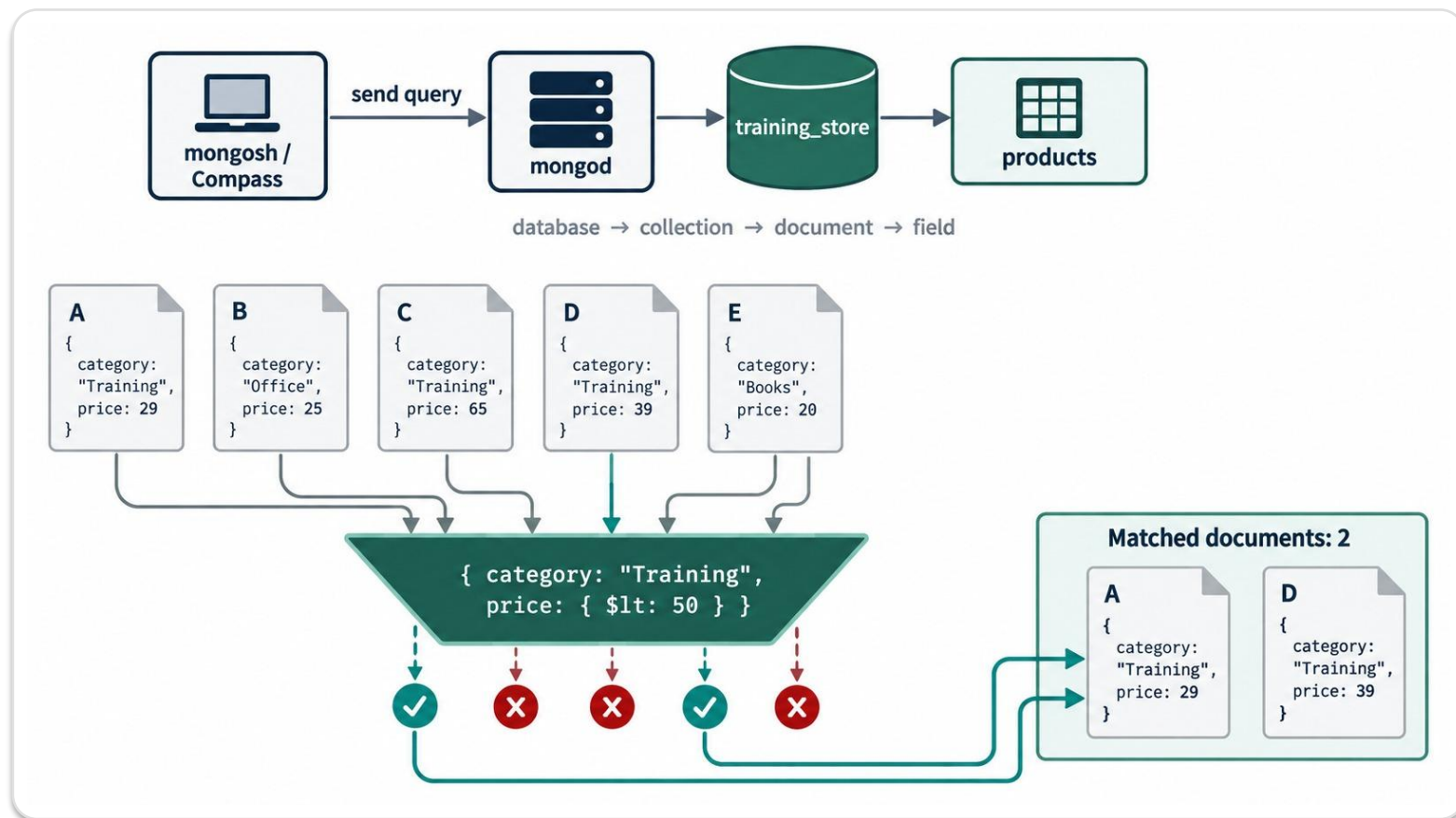
How do we ask for exactly the documents — and fields — we need?

IN THIS PART

- 1 Query filters
- 2 findOne() versus find()
- 3 Filter, projection, sort, limit
- 4 Projection rules
- 5 Sort, skip and limit for pages
- 6 Counting and distinct values

UNDERSTANDING QUERY FILTERS

The filter is a document; each document either matches or doesn't.



A document of conditions

{ field: value } pairs — every pair must be true.

Exact and case-sensitive

"LAPTOP" matches 3 products; "Laptop" matches none.

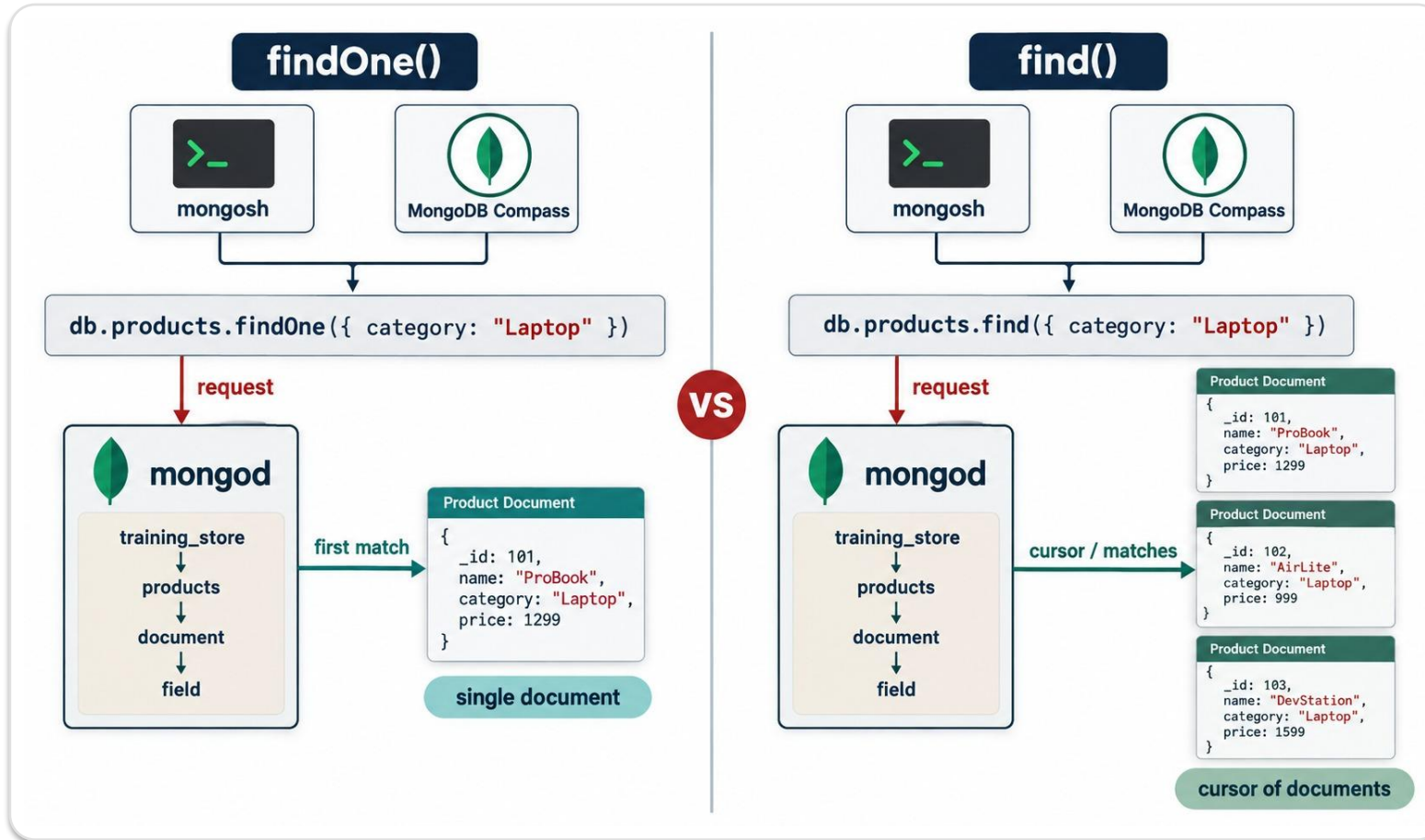
On training_store

```
db.products.find({
  category: "BOOK",
  price: { $lt:
    Decimal128("50.00") }
})
// Query Cookbook (39.99)
```

KEY TAKEAWAY A filter is a document of conditions — a document matches only when every condition is true.

FINDONE() VS FIND()

One document or null — or a cursor over every match.



findOne()

Returns one document, or null when nothing matches.

find()

Returns a cursor; `mongosh` prints 20 at a time — type it for more.

By a unique field

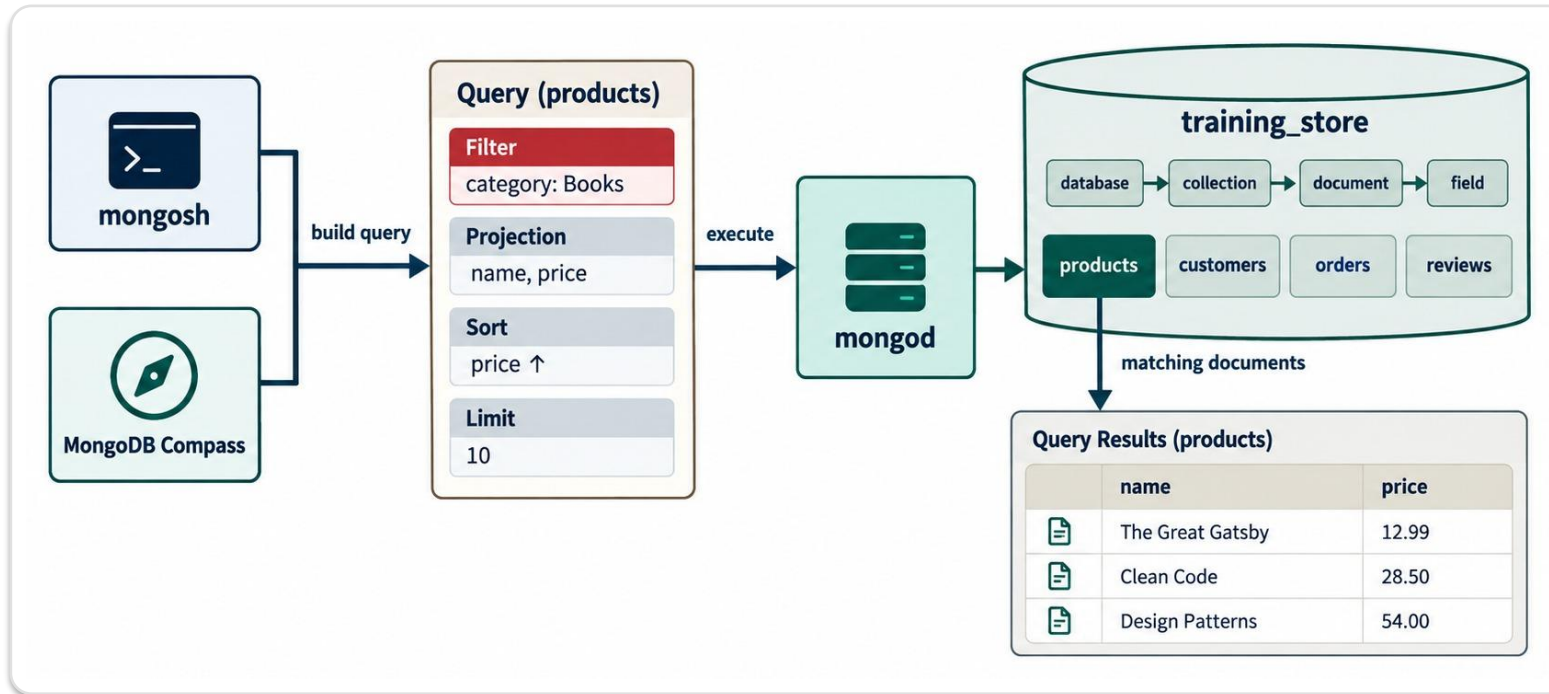
```
db.products.findOne(  
  { sku: "L100" }  
)  
// Business Laptop
```

TIP `sku` has a unique index, so one match is guaranteed.

KEY TAKEAWAY Use `findOne()` when you expect one document and `find()` when you expect a list.

ANATOMY OF A READ QUERY

Filter, projection, sort and limit — the four questions of a read.



The same shape on training_store

```
db.products.find(  
  { category: "BOOK" },  
  { _id: 0, name: 1,  
    price: 1 }  
) .sort({ price: 1 })  
  .limit(10)
```

Result

Query Cookbook 39.99, then MongoDB Fundamentals 59.99.

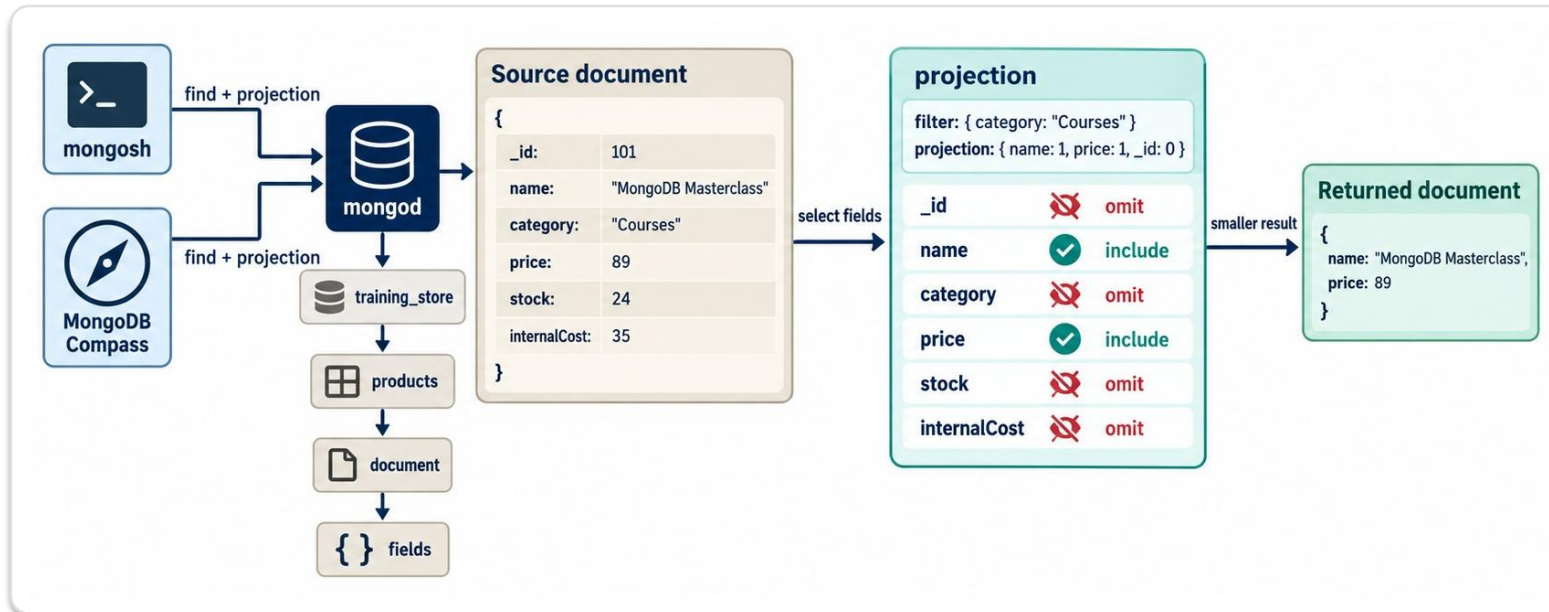
Order of thought

Which documents, which fields, which order, how many?

KEY TAKEAWAY Build a read in four steps: filter the documents, project the fields, sort them, limit the count.

PROJECTION FUNDAMENTALS

The second argument of find() chooses the fields that come back.



Include

`{ name: 1, price: 1 }` — `_id` comes too unless you add `_id: 0`.

Exclude

`{ attributes: 0, tags: 0 }` — every other field comes back.

Don't mix

`{ name: 1, tags: 0 }` is an error; only `_id` may be mixed.

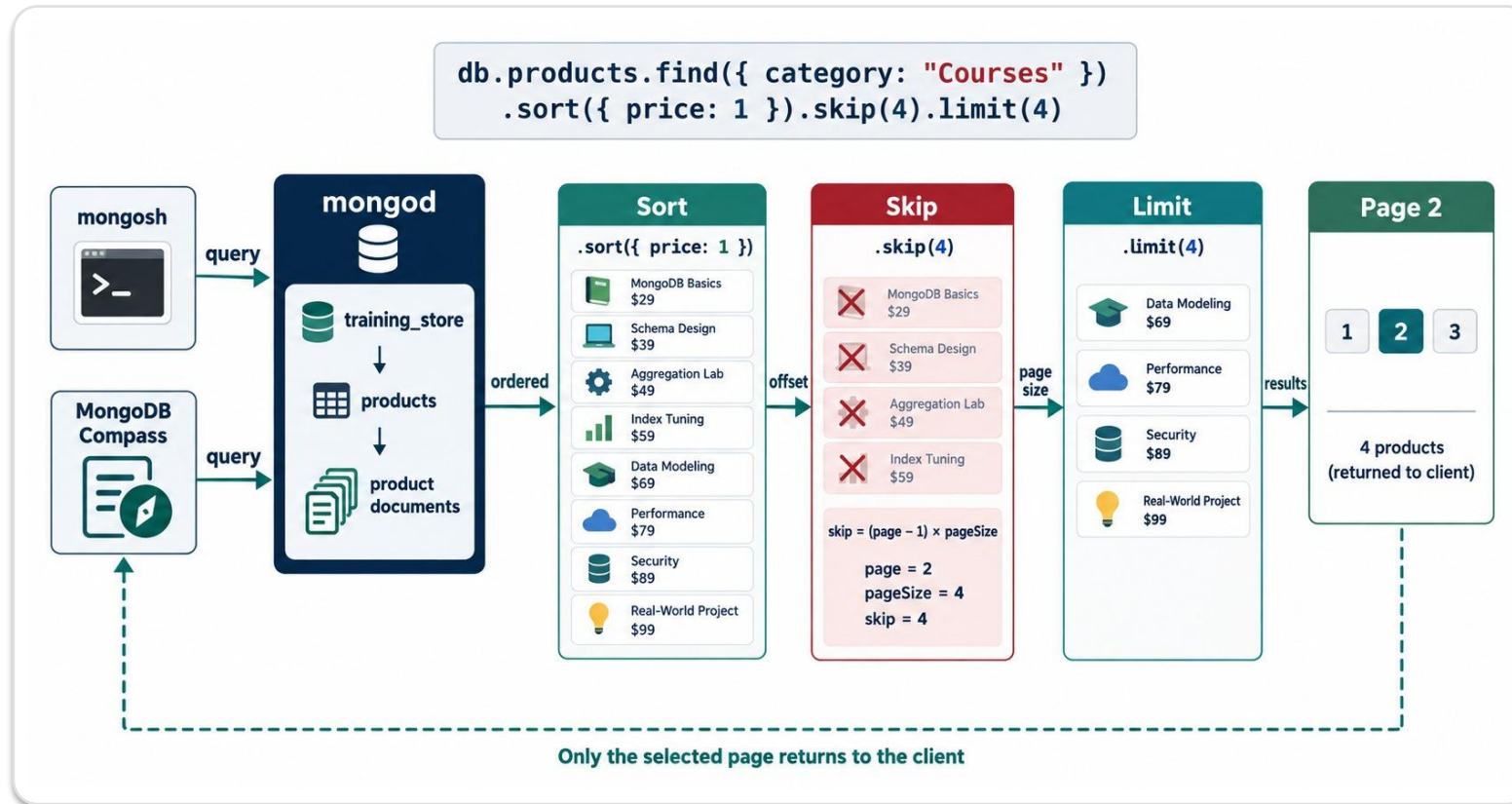
Nested paths work too

```
db.customers.find(  
  { customerNumber: "C101" },  
  { _id: 0, "name.first": 1 })  
// { name: { first: "Aisha" } }
```

KEY TAKEAWAY Return only the fields the caller needs — projection shapes the result, never the stored document.

SORTING, SKIPPING AND LIMITING

sort() orders, skip() jumps, limit() caps — together they make a page.



sort()

1 ascending, -1 descending; add `_id` as a tie-breaker.

skip() and limit()

$\text{skip} = (\text{page} - 1) \times \text{pageSize}$; $\text{limit} = \text{pageSize}$.

Page 2 of active products

```
db.products  
.find({ active: true })  
.sort({ price: 1, _id: 1 })  
.skip(5).limit(5)
```

RESULT B300, S210, S200, A500, L100.

KEY TAKEAWAY Always sort before you page — without a stable order, pages can repeat or skip documents.

COUNTING AND DISTINCT VALUES

How many documents match — and which values a field holds.

Count accurately

```
db.products
  .countDocuments({})
// 13
db.products.countDocuments(
  { active: true })
// 12 — L190 is inactive
```

countDocuments()

Exact count for a filter — your safety check.

Estimate from metadata

```
db.products
  .estimatedDocumentCount()
// 13 — takes no filter

// legacy, avoid:
db.products.find().count()
```

estimatedDocumentCount()

Fast total from metadata, no filter.

Unique values

```
db.products.distinct(
  "category")
// [ "ACCESSORY", "BOOK",
//   "LAPTOP", "SHOE" ]
db.products.distinct(
  "attributes.connection",
  { category: "ACCESSORY" })
// [ "Bluetooth", "USB-C" ]
```

distinct()

Unique values; an optional filter narrows them.

KEY TAKEAWAY Count with countDocuments() before and after every change — it is the cheapest check you have.

EXERCISE: PREDICT THE SORTED RESULT

Read the query, write your answer, then run it.

PRACTICE EXERCISE

10 MIN

SCENARIO

The five accessories cost 19.99, 24.99, 49.99 and 249.99 — and XBAD, whose price is stored as the string "49.99".

```
db.products.find(
  { category: "ACCESSORY" },
  { _id: 0, sku: 1, price: 1 }
).sort({ price: -1 })
.limit(3)
```

YOUR TASKS

- 1 Write the three SKUs you expect, in order
- 2 Say where XBAD lands, and why
- 3 Run it on the instructor's screen
- 4 Change the filter so only Decimal128 prices sort

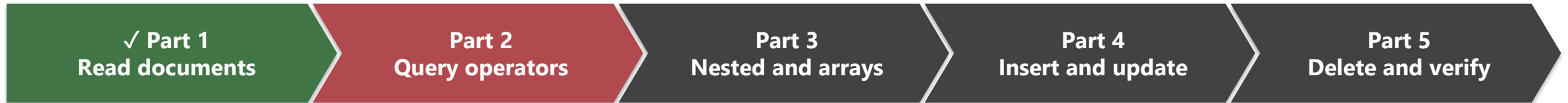
EXPECTED OUTCOME

- ✓ XBAD first: strings sort after numbers
- ✓ Then A500 (249.99) and P1001 (49.99)
- ✓ Add price: { \$type: "decimal" }
- ✓ Then A500, P1001, A400

KEY TAKEAWAY Mixed BSON types sort by type first, then value — one bad document can top a price list.

PART 2: QUERY OPERATORS

Comparison, logical and element operators.



SO FAR

- ✓ Filters are documents of conditions
- ✓ Projection, sort and limit shape the result
- ✓ `countDocuments()` checks every step

NEXT QUESTION

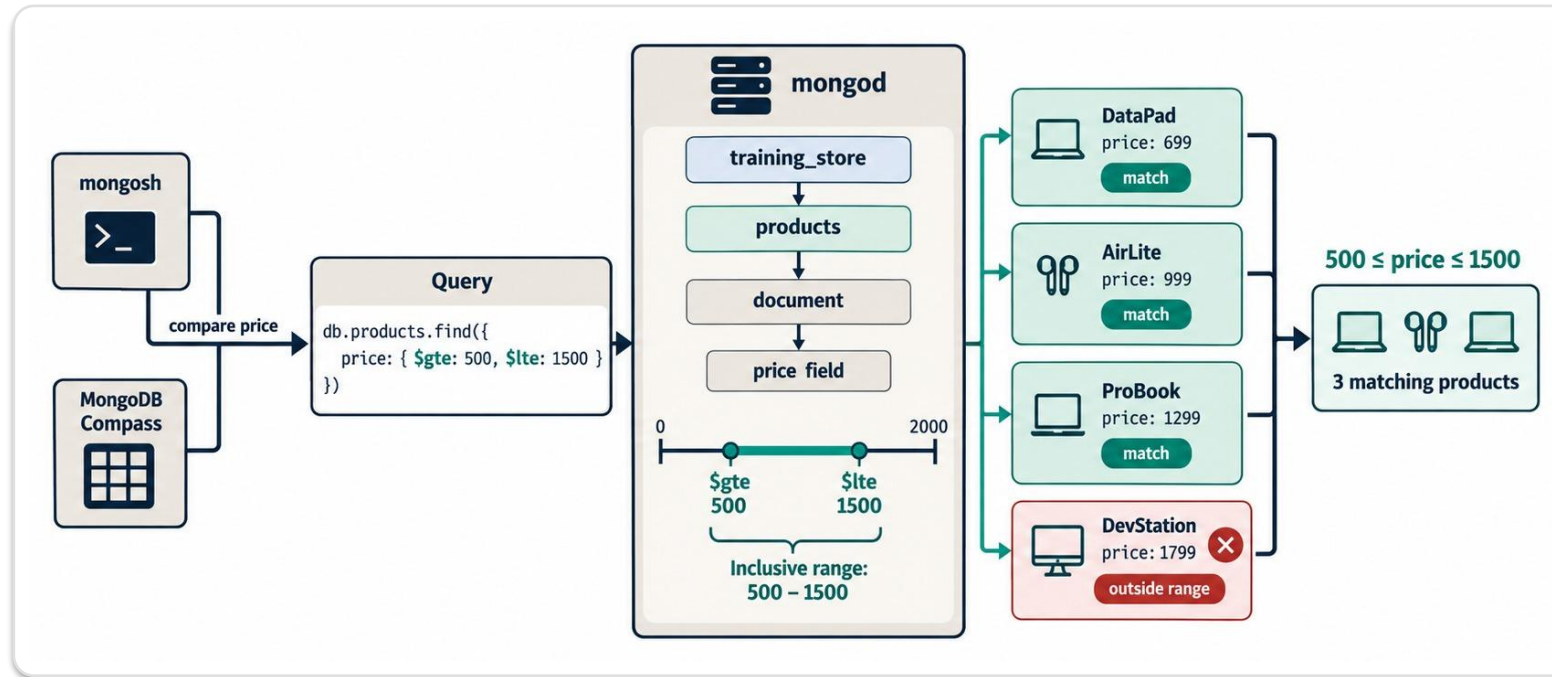
How do we express ranges, lists, either-or and missing values?

IN THIS PART

- 1 Comparison operators and ranges
- 2 `$in` and `$nin`
- 3 `$and` and `$or`
- 4 Building a filter step by step
- 5 `$not` and `$nor`
- 6 Missing, null and BSON type
- 7 Regular expressions and dates

COMPARISON OPERATORS

\$gt, \$gte, \$lt and \$lte — inside the field they test.



Inside the field

`{ price: { $gt: ... } }` — never `{ $gt: { price: ... } }`.

Money is Decimal128

Compare with `Decimal128("100.00")`; a string never matches a number.

A range

Two operators on one field; `$gte` and `$lte` include both ends.

ON TRAINING_STORE 50.00 to 500.00
returns L190, S200, S210, B300 and A500.

KEY TAKEAWAY Put comparison operators inside the field, and compare with the same BSON type that is stored.

COMPARISONS, \$IN AND \$NIN ON TRAINING_STORE

Real results — and the product that never shows up.

Above 100

```
db.products.find(
  { price: { $gt:
    Decimal128("100.00") } },
  { _id: 0, sku: 1 })
// L100 L110 L190
// S200 A500
```

XBAD is missing above

Its price is a string, so no numeric range sees it.

Any value in a list

```
db.products.countDocuments({
  category: { $in:
    ["LAPTOP", "BOOK"] } })
// 5
db.reviews.countDocuments(
  { rating: { $gte: 4 } })
// 5 of 6 reviews
```

\$in

An OR on one field — shorter than \$or.

None of a list

```
db.products.find(
  { category: { $nin:
    ["LAPTOP", "SHOE"] } },
  { _id: 0, sku: 1 })
// B300 B310 P1001 A400
// A410 A500 XBAD
```

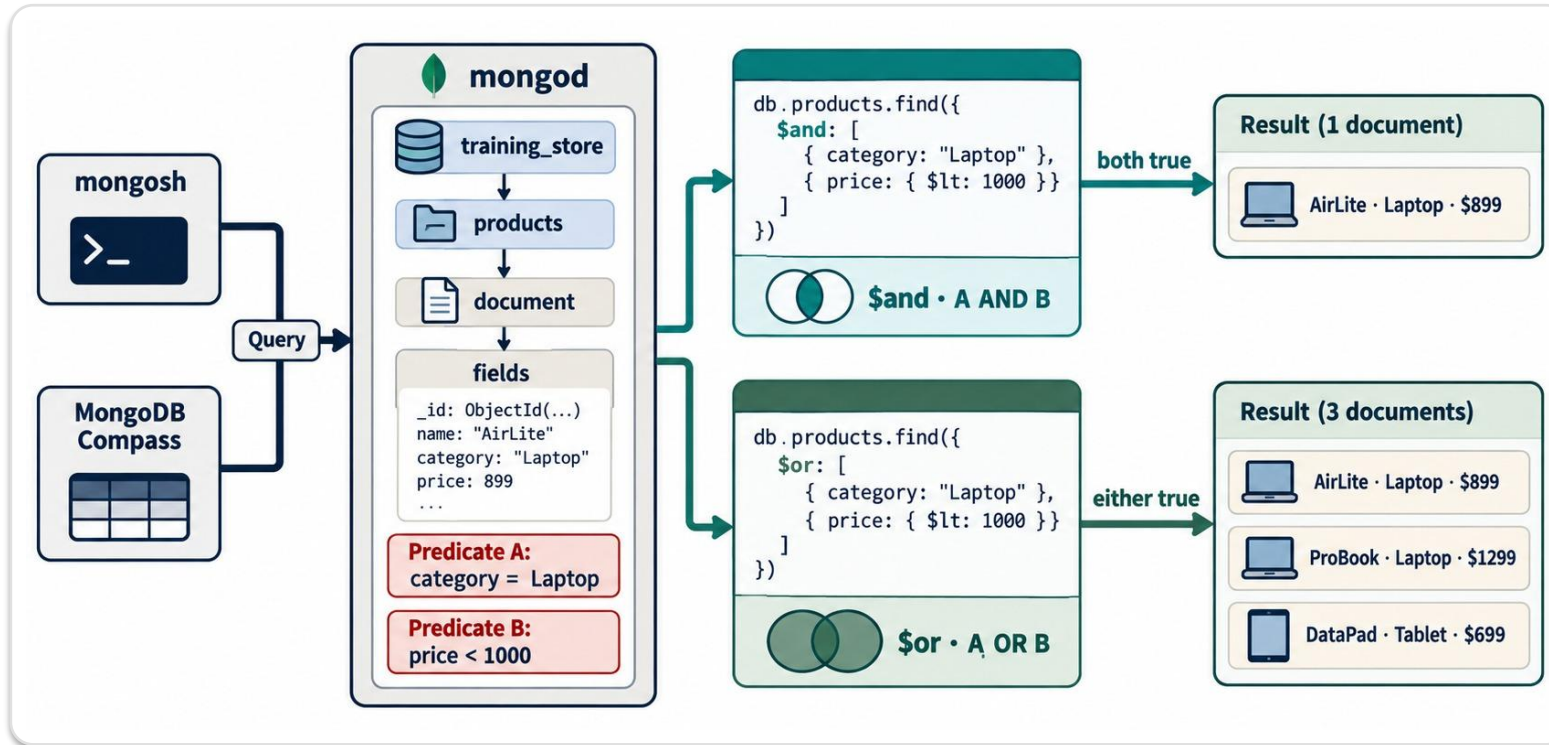
\$nin and \$ne

Also match documents where the field is missing.

KEY TAKEAWAY Use \$in and \$nin for lists on one field — and remember that negative operators also match missing fields.

LOGICAL OPERATORS: \$AND AND \$OR

Both conditions true — or at least one.



AND

Commas mean AND; write \$and only when a field or operator repeats.

\$or

Any condition true. Same field? Use \$in instead.

Two different fields: \$or

```
db.orders.find({ $or: [
  { paymentStatus: "PENDING" },
  { fulfillmentStatus:
    "PROCESSING" } ] })
// 05001 06202 06302 06401 06499
```

KEY TAKEAWAY Write AND with commas, OR with \$or across fields, and \$in when the alternatives share one field.

BUILD A FILTER ONE CONDITION AT A TIME

Demo 4.2 on products: add one condition, count, and explain the number.

FILTER SO FAR

COUNT · WHO IS LEFT

```
{ }
```

13

Every product

```
{ category: "ACCESSORY" }
```

5

P1001, A400, A410, A500, XBAD

```
{ category: "ACCESSORY",  
  active: true }
```

5

All five accessories are active

```
{ category: "ACCESSORY",  
  active: true,  
  price: { $lte:  
    Decimal128("100.00") } }
```

3

P1001, A400, A410

Why one at a time

When the result is wrong, you know which condition caused it.

Count after each step

```
db.products.countDocuments({  
  category: "ACCESSORY" }) // 5
```

Two drops explained

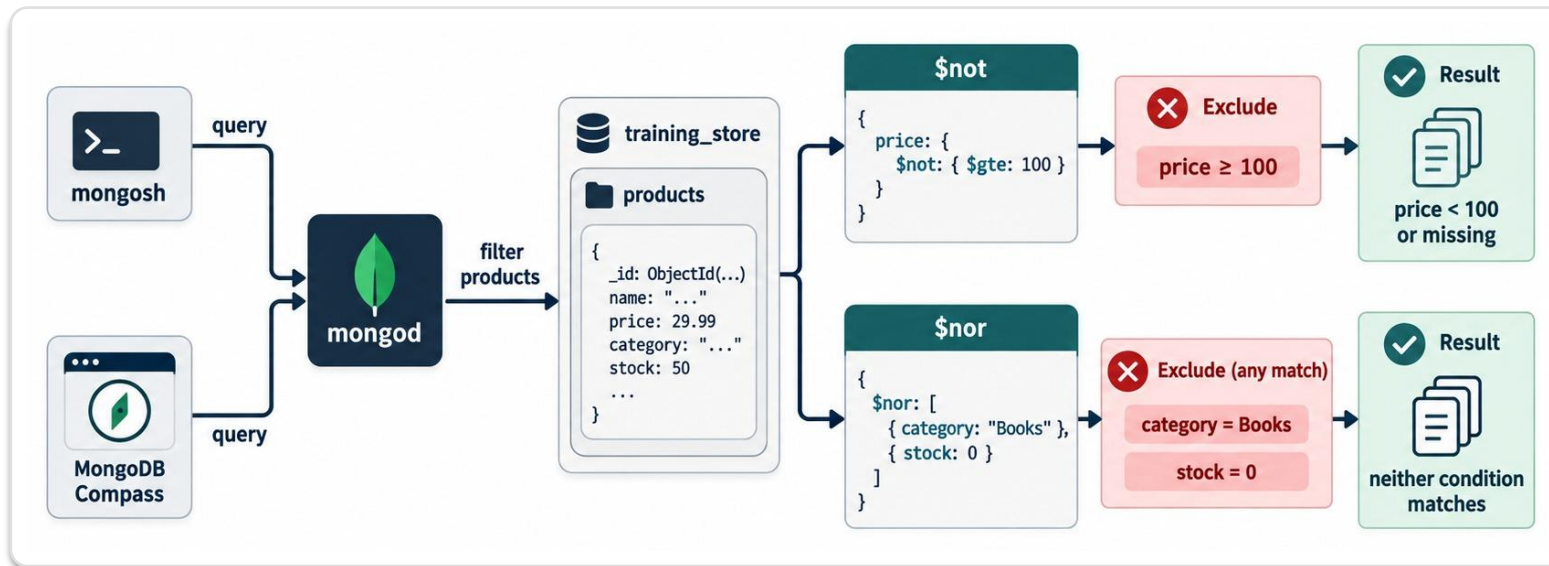
A500 costs 249.99. XBAD's price is the string "49.99", so a Decimal128 range skips it.

SURPRISED? Check the field name, case and BSON type first.

KEY TAKEAWAY Grow a filter one condition at a time and count after each step — every drop should have a reason.

THE \$NOT AND \$NOR OPERATORS

Negate one expression — or reject every listed condition.



\$not

Wraps an operator: `{ price: { $not: { $gt: ... } } }`.
Missing or non-numeric prices pass too.

\$nor

Keeps documents that fail every listed condition.

Prefer \$ne

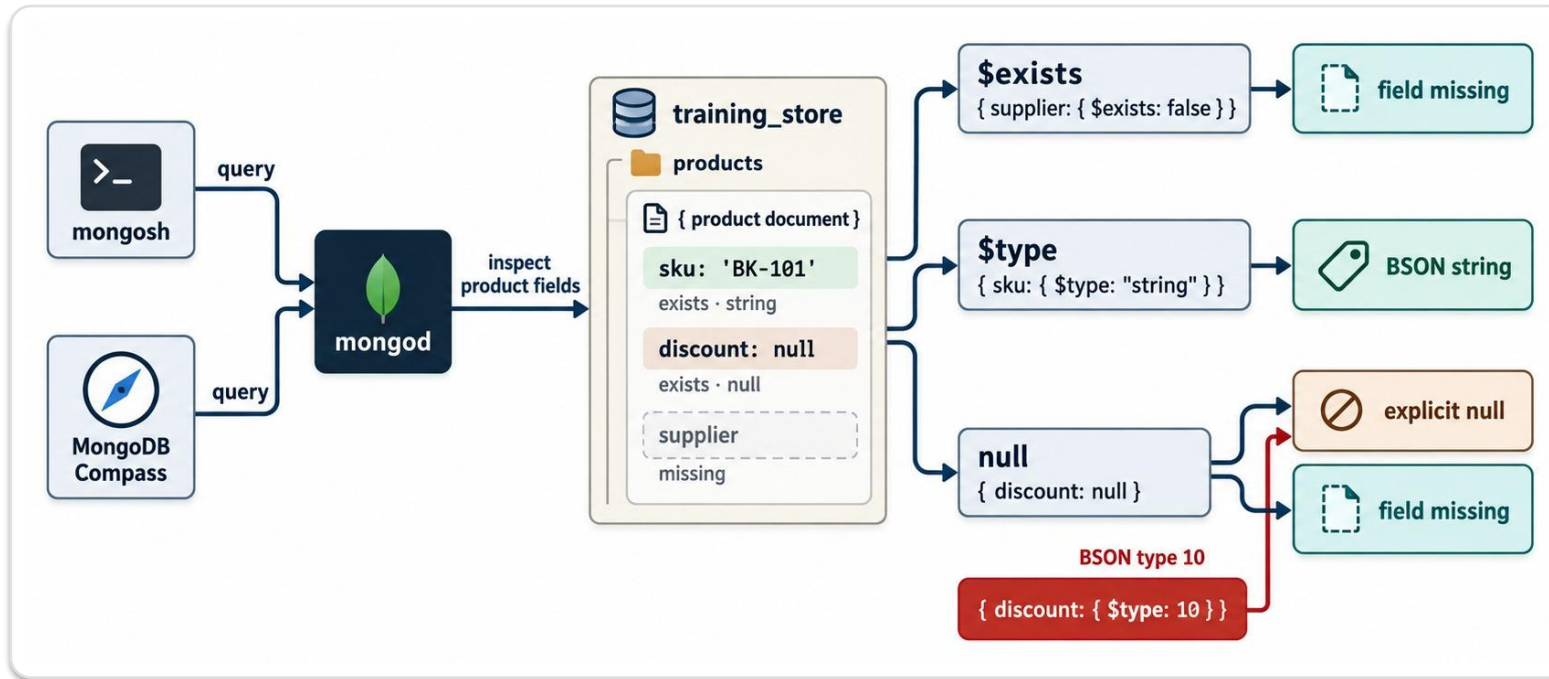
For a simple not-equal, `$ne` reads better than `$not`.

ON TRAINING_STORE `$nor: [{ category: "BOOK" }, { active: false }]` keeps 10 of 13 products.

KEY TAKEAWAY Negations over-match easily: `$not`, `$nor`, `$ne` and `$nin` all keep documents where the field is missing.

FIELD EXISTENCE, BSON TYPE AND NULL

Missing, explicit null and the wrong type are three different questions.



{ field: null }

Explicit null or missing: C204 and C515.

\$type: "null"

Explicit null only: C204's phone.

\$exists: false

Missing only: C515 has no phone field.

Find the planted type error

```
db.products.find(
  { price: { $type:
    "string" } },
  { _id: 0, sku: 1 })
// { sku: "XBAD" }
```

KEY TAKEAWAY Ask exactly what you mean: \$exists for presence, \$type for the BSON type, null for null-or-missing.

REGULAR EXPRESSIONS AND DATE RANGES

Match text by pattern; match time with a start and an end.

Pattern match on a string

```
db.products.find(
  { name: { $regex: "^MongoDB" } },
  { _id: 0, name: 1 })
// MongoDB Fundamentals

db.products.countDocuments({
  name: { $regex: "wireless",
    $options: "i" } })
// 2 – Keyboard and Mouse
```

^ anchors

Starts with: precise, and can use an index.

The i option

Case-insensitive; unanchored patterns scan every value.

✓ Half-open date range

```
db.orders.find(
  { createdAt: {
    $gte: ISODate("2026-09-19T00:00:00Z"),
    $lt:  ISODate("2026-09-20T00:00:00Z")
  } },
  { _id: 0, orderNumber: 1 })
// 06402, 06499: every order
// placed on 19 September (UTC)
```

Half-open range

\$gte the start, \$lt the next day: no time-of-day gaps.

KEY TAKEAWAY Anchor regular expressions when you can, and query dates from an inclusive start to an exclusive end.

EXERCISE: TURN A BUSINESS QUESTION INTO A FILTER

From a merchandising request to a working query.

PRACTICE EXERCISE

10 MIN

SCENARIO

Merchandising asks: which active products are on clearance or premium? They want the SKU and name only.

```
db.products.find(  
  { /* your filter */ },  
  { /* your projection */ }  
)
```

YOUR TASKS

- 1 Choose the fields and operators
- 2 Write the filter and the projection
- 3 Predict the SKUs before you run it
- 4 Explain why L190 is not in the result

EXPECTED OUTCOME

- ✓ active: true plus tags: { \$in: [...] }
- ✓ { _id: 0, sku: 1, name: 1 }
- ✓ L110, S290 and A500
- ✓ L190 is clearance but inactive

KEY TAKEAWAY Translate each phrase of the request into one condition — then check the result against the data.

PART 3: NESTED FIELDS AND ARRAYS

Query inside embedded documents and arrays.



SO FAR

- ✓ Comparison and logical operators
- ✓ Missing, null and type are different
- ✓ Negations also match missing fields

NEXT QUESTION

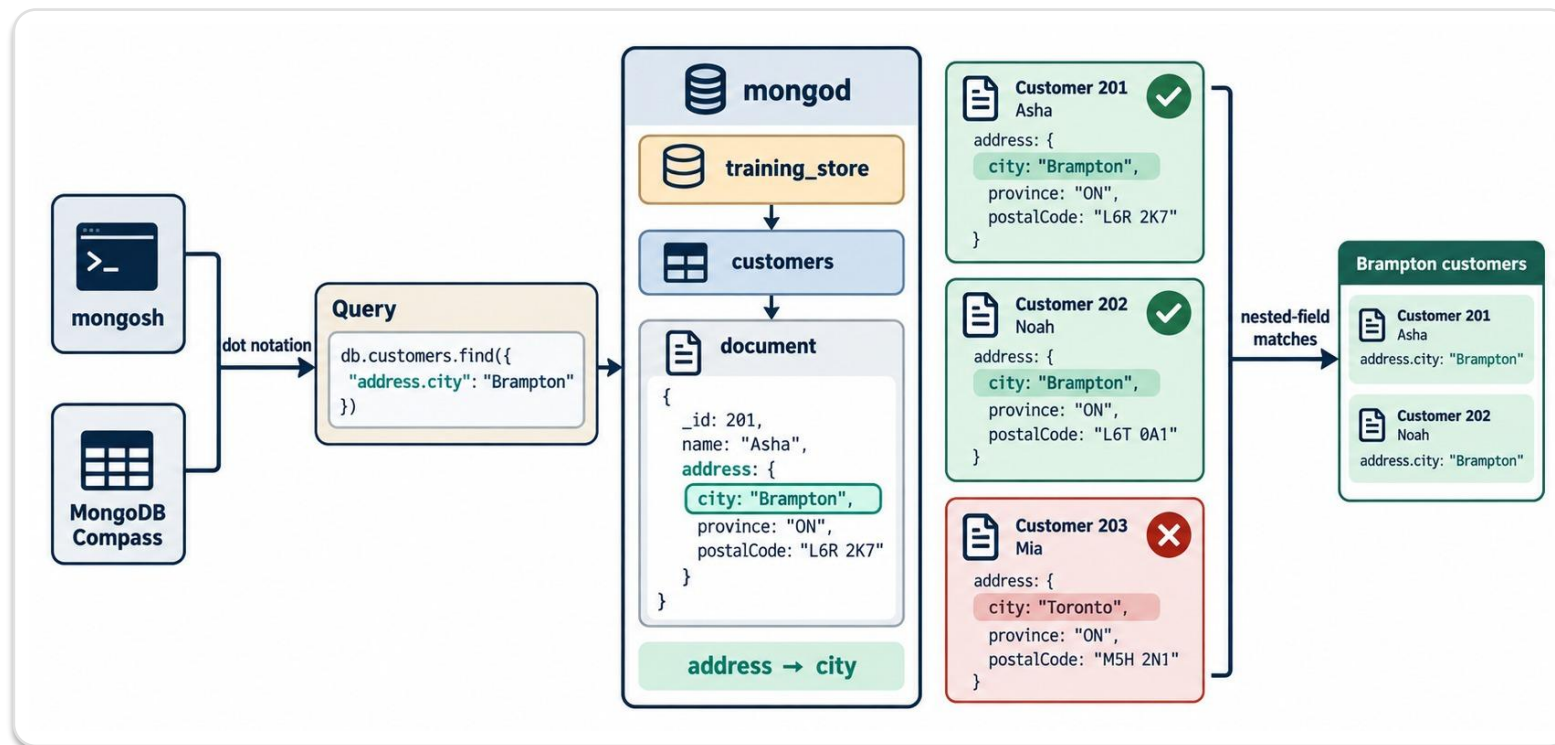
How do we reach inside embedded documents and arrays — and match the right element?

IN THIS PART

- 1 Dot notation for embedded fields
- 2 Contains, exact match, \$all and \$size
- 3 \$elemMatch on arrays of documents
- 4 Practice: choose the array query

QUERYING NESTED FIELDS

Dot notation reaches inside embedded documents.



Quote the path

"attributes.memoryGB" — unquoted dots are a syntax error.

Through arrays too

"addresses.city" looks at every address in the array.

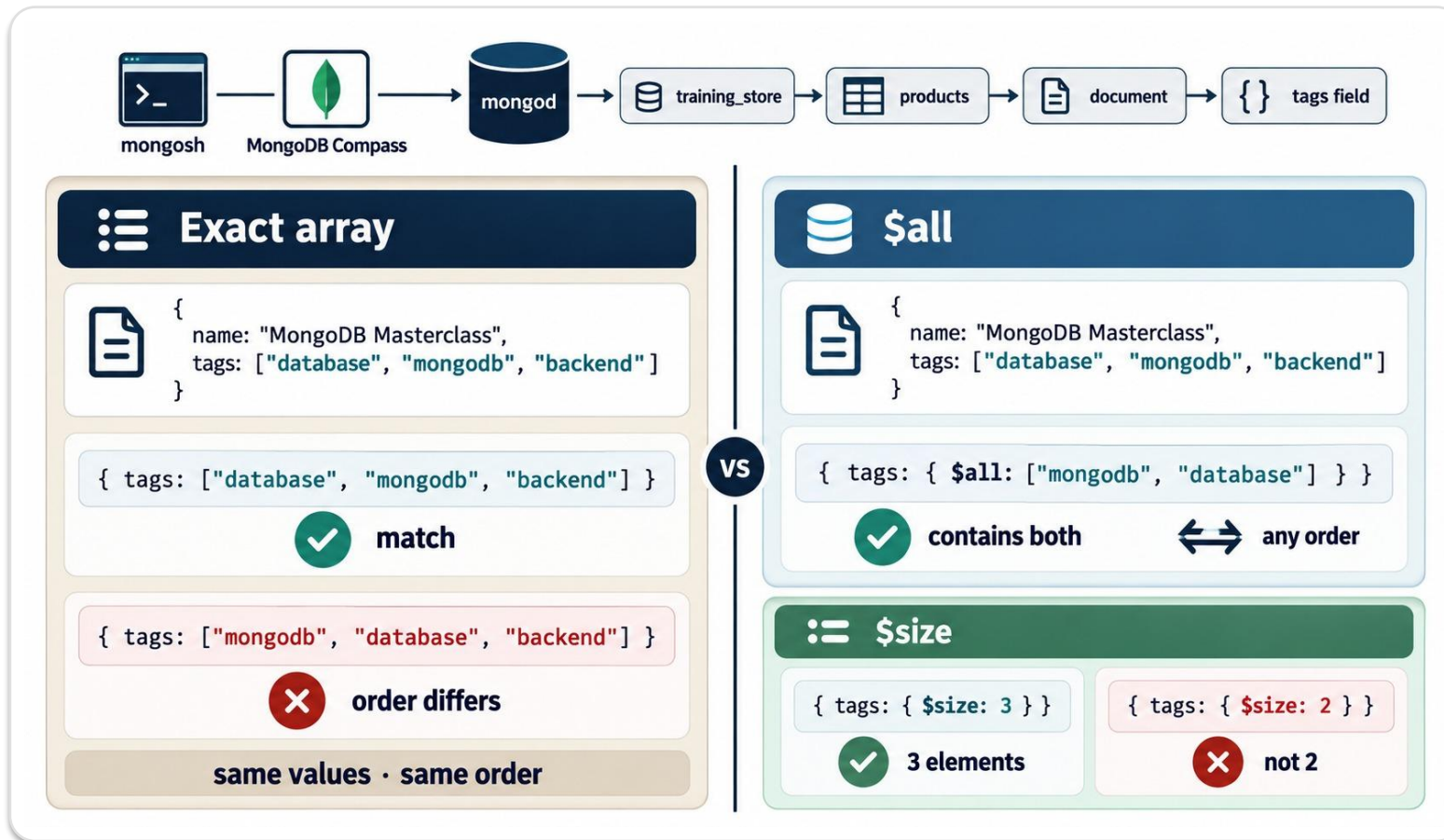
On training_store

```
db.customers.find(
  { "addresses.city": { $in:
    ["Toronto", "Montreal"] } },
  { _id: 0, customerNumber: 1 })
// C101, C412
```

KEY TAKEAWAY Dot notation reaches into embedded documents and arrays — quote the path and match the stored value exactly.

EXACT ARRAY MATCH VS \$ALL VS \$SIZE

Contains a value, equals the array, contains all, or has N elements.



Contains

{ tags: "wireless" } — any element: P1001, A410.

Exact array

["database", "technology"] → B300; reversed order finds nothing.

\$all

All listed values, any order: ["office", "premium"] → A500.

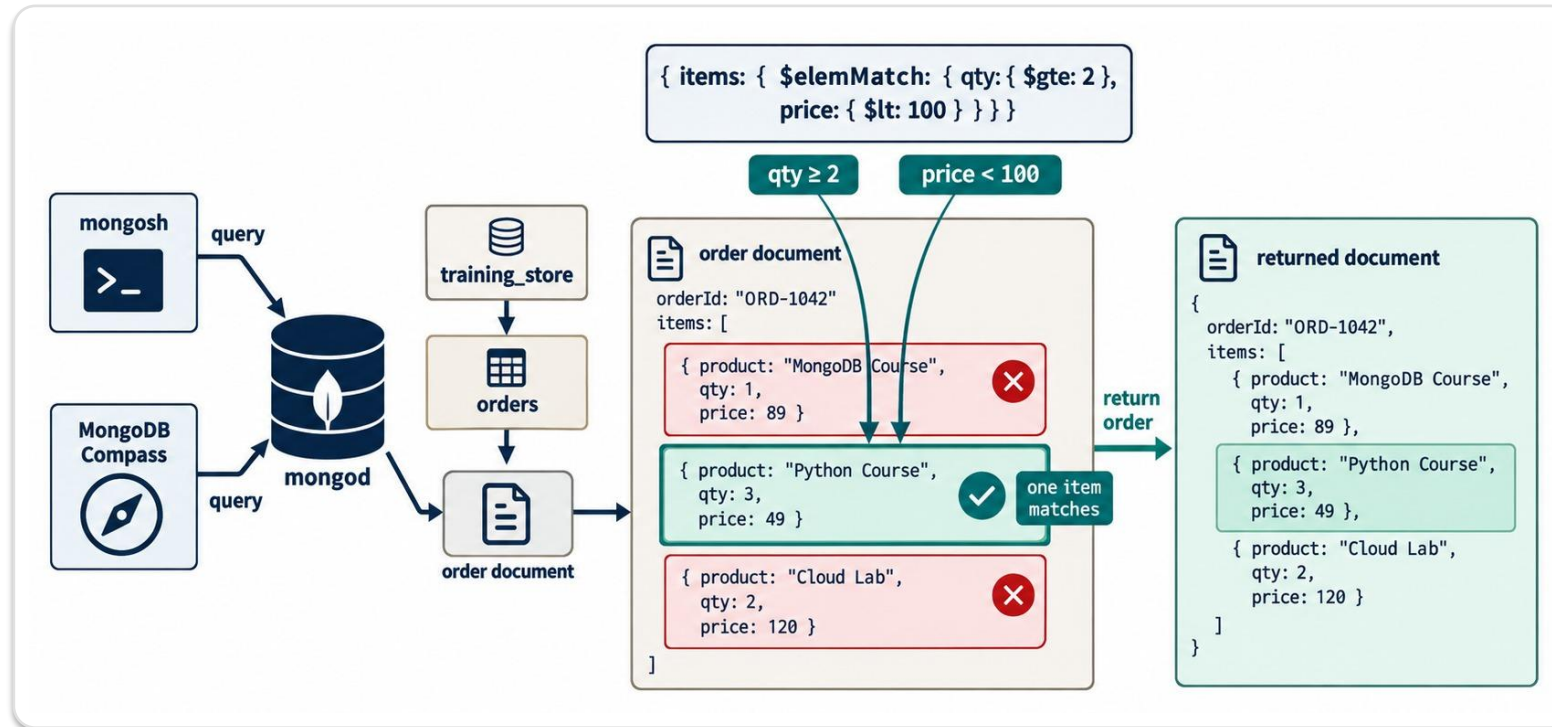
\$size

Exact length only: \$size: 3 → L190. No "at least".

KEY TAKEAWAY Match one element by value and several with \$all; use exact arrays or \$size only when order or length matters.

QUERYING ARRAYS OF DOCUMENTS WITH \$ELEMATCH

Every condition must be met by the same array element.



Without \$elemMatch

"items.sku" and "items.quantity" can be met by different items.

The O5001 trap

L100 × 1 plus A410 × 2 — dot notation matches it; `$elemMatch` doesn't.

Same item, both conditions

```
db.orders.find({ items: {
  $elemMatch: {
    sku: "L100",
    quantity: { $gte: 2 }
  } })
// only 06401 (L100 × 2)
```

KEY TAKEAWAY When several conditions must hold for the same array element, wrap them in `$elemMatch`.

EXERCISE: CHOOSE THE RIGHT ARRAY QUERY

Four requests, four different array tools.

PRACTICE EXERCISE

10 MIN

SCENARIO

The store team sends four requests. Pick containment, dot notation, \$all or \$elemMatch for each.

- A Shoes available in size 9
- B Orders with L100 x 2 or more
- C Customers in Toronto or Montreal
- D Products tagged office AND premium

YOUR TASKS

- 1 Choose the operator for each request
- 2 Write each filter
- 3 Predict the matching documents
- 4 Say which one dot notation gets wrong

EXPECTED OUTCOME

- ✓ A "attributes.sizes": 9 → S200, S210, S290
- ✓ B items \$elemMatch → O6401
- ✓ C "addresses.city" \$in → C101, C412
- ✓ D tags \$all → A500
- ✓ B: dot notation adds O5001

KEY TAKEAWAY Choose the array tool from the question: one value, all values, a nested path, or one element meeting several conditions.

PART 4: INSERTS, UPDATES, UPSERTS AND REPLACE

Change documents — and only the fields you mean.



SO FAR

- ✓ Filters find exactly the right documents
- ✓ Dot notation and \$elemMatch reach inside
- ✓ Every read can be counted first

NEXT QUESTION

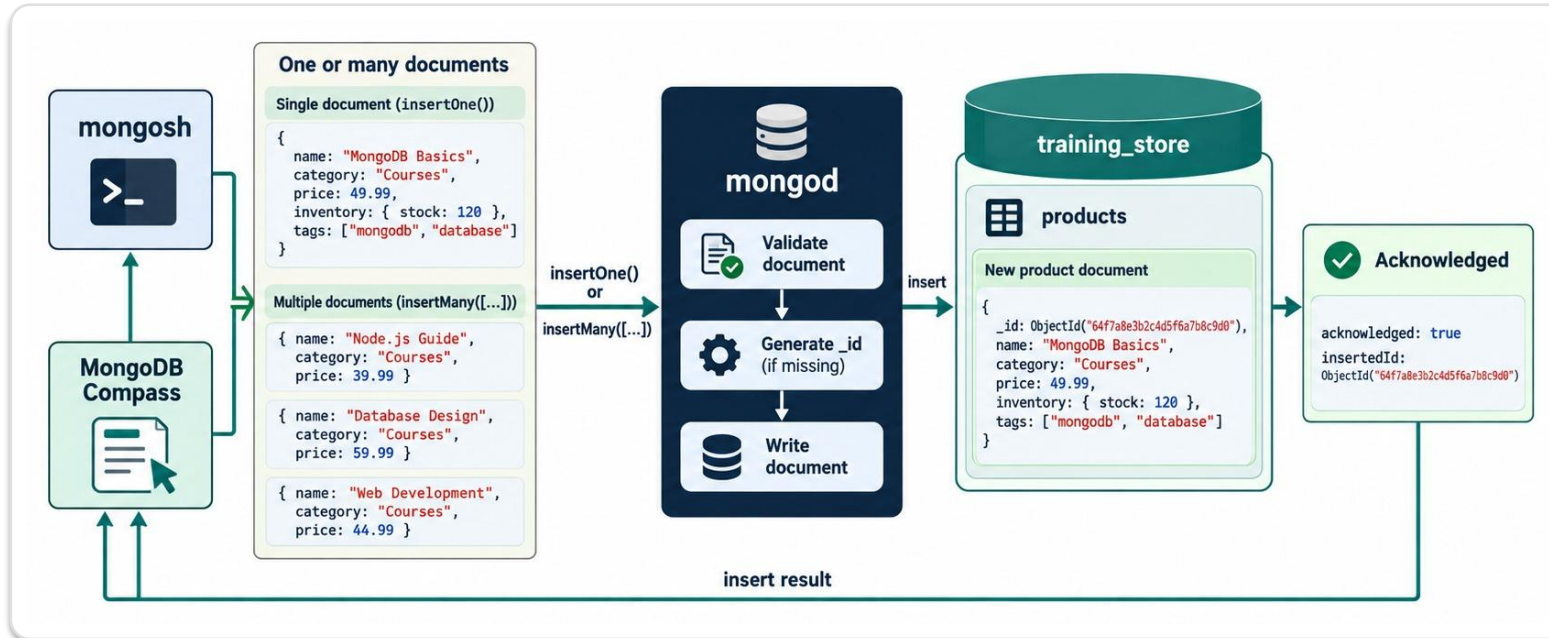
How do we add and change documents without losing data we meant to keep?

IN THIS PART

- 1 insertOne() and insertMany()
- 2 updateOne() and updateMany()
- 3 \$set, \$unset, \$inc and \$rename
- 4 Array updates and positional \$
- 5 replaceOne() versus \$set
- 6 Upserts with \$setOnInsert
- 7 Reading write results

INSERTING DOCUMENTS

insertOne() or insertMany(); mongod adds _id when it is missing.



Demo 4.5

```
db.products.insertOne({
  sku: "DEMO-1", name: "Demo Cable",
  category: "ACCESSORY",
  price: Decimal128("9.99"),
  active: true })
// acknowledged: true,
// insertedId: ObjectId(...)
```

insertMany()

Returns insertedIds, one per document.

Unique sku

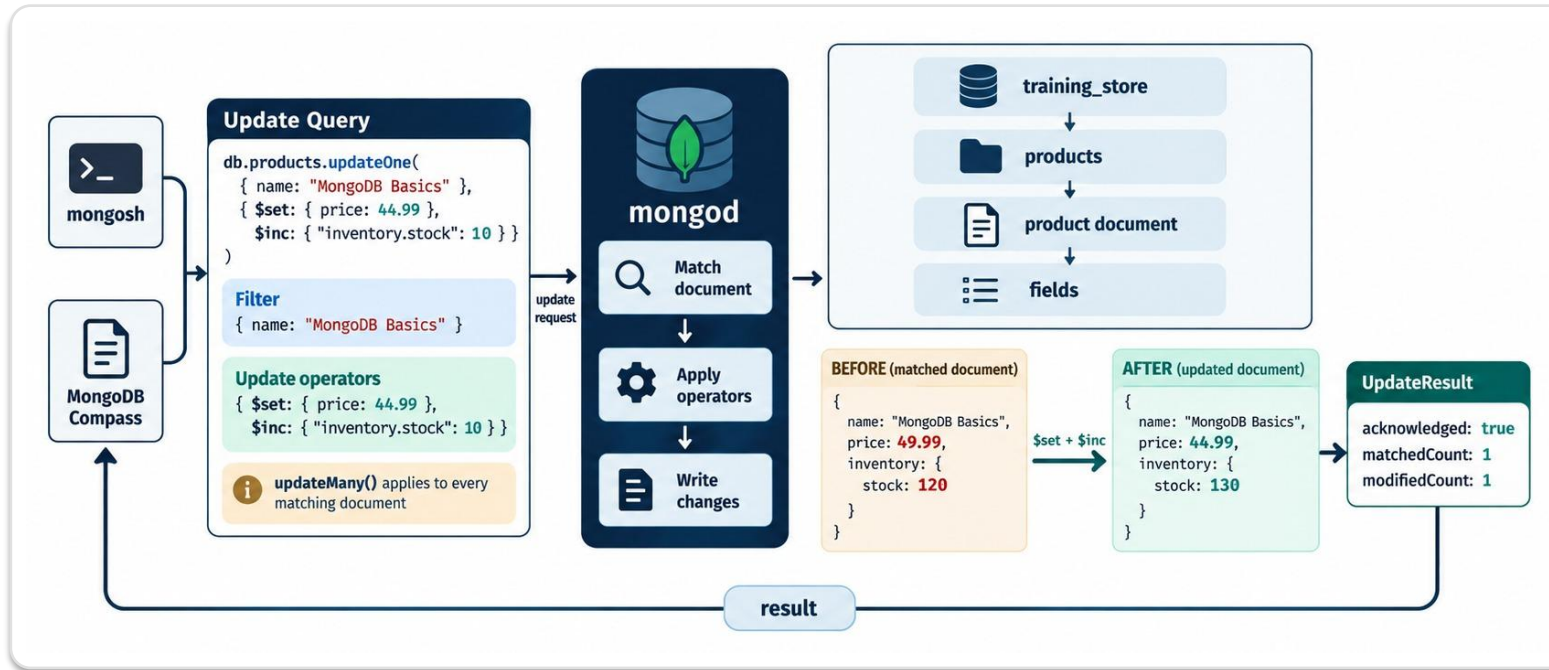
A second DEMO-1 fails with E11000 duplicate key.

VERIFY findOne({ sku: "DEMO-1" }), then delete it.

KEY TAKEAWAY Insert with real BSON types — Decimal128 for money, Date for time — and read the document back to verify.

UPDATING DOCUMENTS

A filter, update operators and a write result.



Three parts

`updateOne(filter, update, options)`: the filter picks, the operators change.

One or many

`updateMany()` changes every match — preview it with `find()` first.

Already true?

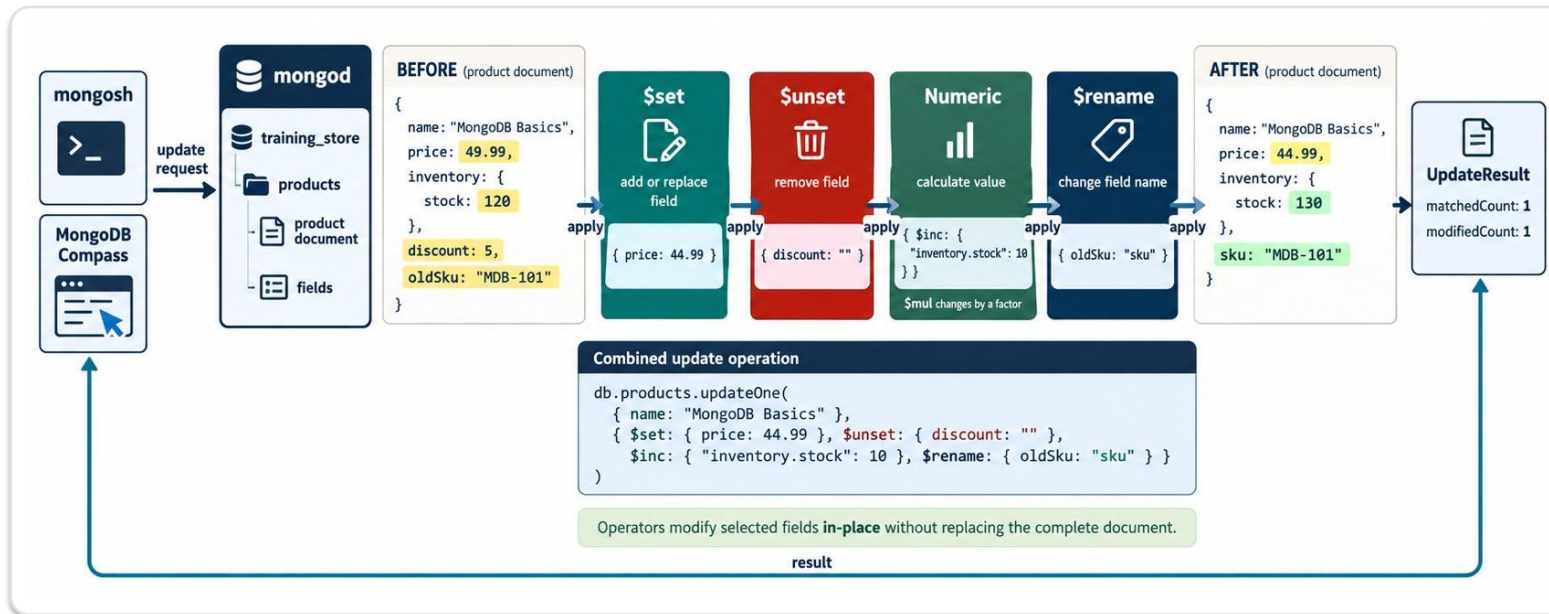
```
db.products.updateOne(
  { sku: "L100" },
  { $set: { active: true } })
// matchedCount: 1,
// modifiedCount: 0
```

WHY 0 L100 is already active: found, nothing to change.

KEY TAKEAWAY An update is a filter plus operators — the result says what was found and what actually changed.

\$SET, \$UNSET, NUMERIC OPERATORS AND \$RENAME

Four kinds of operator, one document, changed in place.



\$set / \$unset

Add or replace a field; remove one — A410's temporaryNote.

\$inc / \$mul

Change a number in place, atomically.

\$min / \$max

Change only if the new value is lower, or higher.

\$rename

legacyName → catalogName on L190. Preview the filter first.

KEY TAKEAWAY Update operators change only the fields you name — the rest of the document stays as it was.

NESTED \$SET AND A GUARDED \$INC

Change one embedded field; decrement stock only when there is enough.

✓ Set one nested field

```
db.products.updateOne(  
  { sku: "L100" },  
  { $set: {  
    "attributes.warrantyYears": 2  
  } })  
// memoryGB, processor ... kept
```

✓ Decrement only if stock allows

```
// Lab 3 first sets stockQuantity: 18  
db.products.updateOne(  
  { sku: "A410",  
    stockQuantity: { $gte: 2 } },  
  { $inc: { stockQuantity: -2 },  
    $currentDate: { updatedAt: true } })  
// 18 → 16
```

Dotted path

`$set: { contact: { ... } }` would replace the object and lose phone.

Atomic

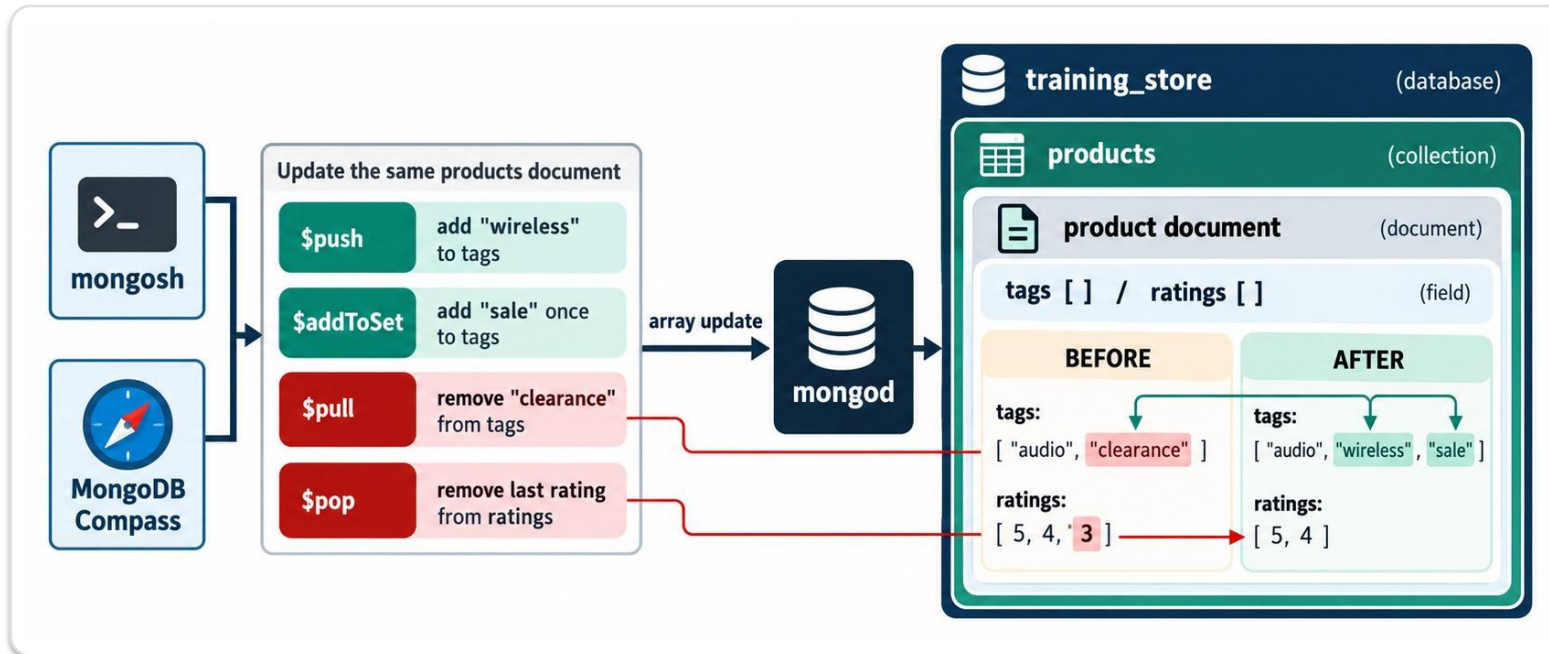
Check and change happen in one single-document write.

MATCHEDCOUNT: 0 Not enough stock — or no such SKU.

KEY TAKEAWAY Set nested fields by their dotted path, and put the business rule in the filter so check and change are one write.

ARRAY UPDATE OPERATORS

\$push, \$addToSet, \$pull and \$pop change arrays without rewriting them.



\$push

Appends — run it twice and the value is there twice.

\$addToSet

Adds only if absent; \$each adds several.

\$pull / \$pop

Remove by value or condition / from one end.

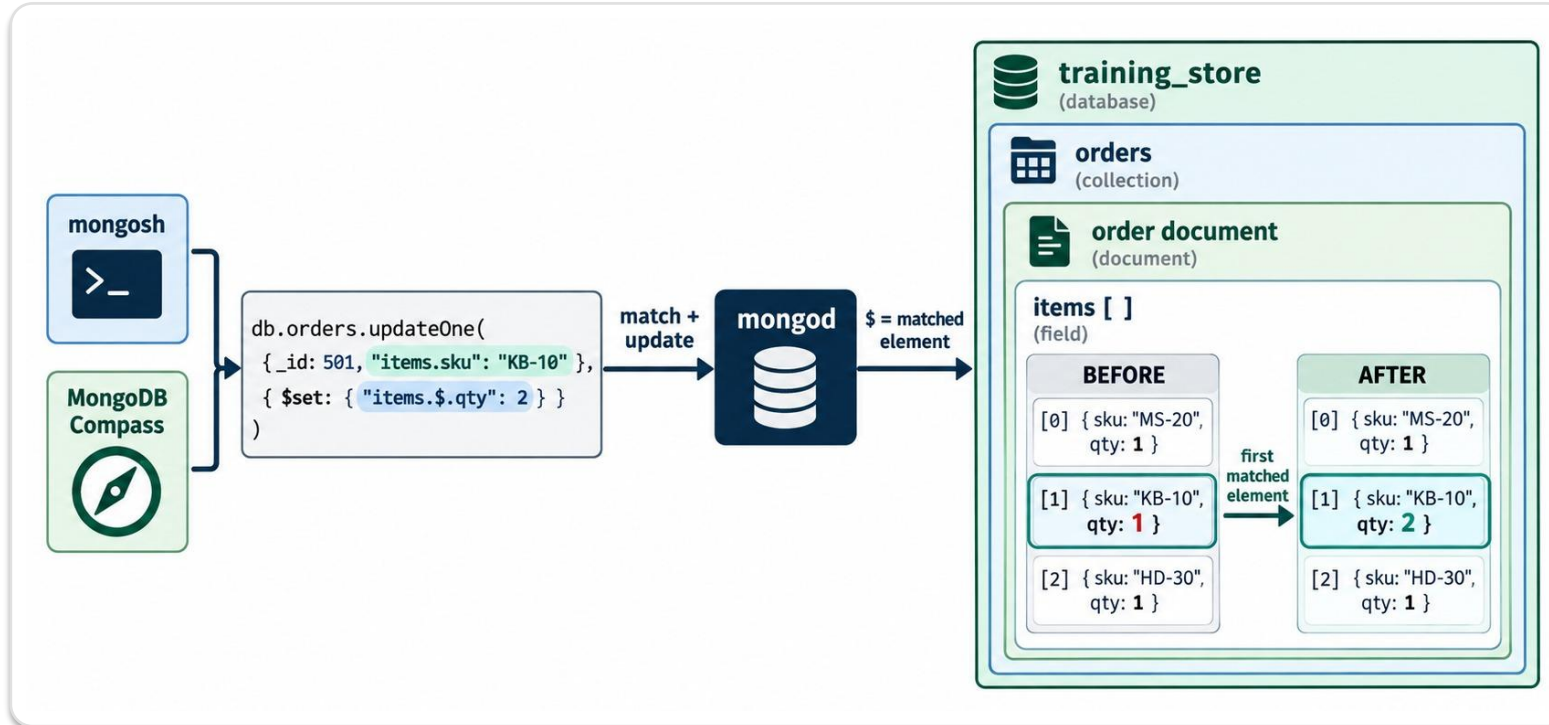
Lab 3: wireless stays once

```
db.products.updateOne(
  { sku: "A410" },
  { $addToSet: { tags: { $each:
    ["featured", "office", "wireless"]
  } } })
```

KEY TAKEAWAY Use \$addToSet when duplicates are wrong, \$push when repeats and order matter, and \$pull to remove by value.

POSITIONAL ARRAY UPDATES

\$ updates the element the filter matched.



\$

The first element the filter matched:
items.\$.quantity.

[\$]

Every element: items[\$].reviewed.

[\$[name] + arrayFilters]

Only the elements that pass a condition.

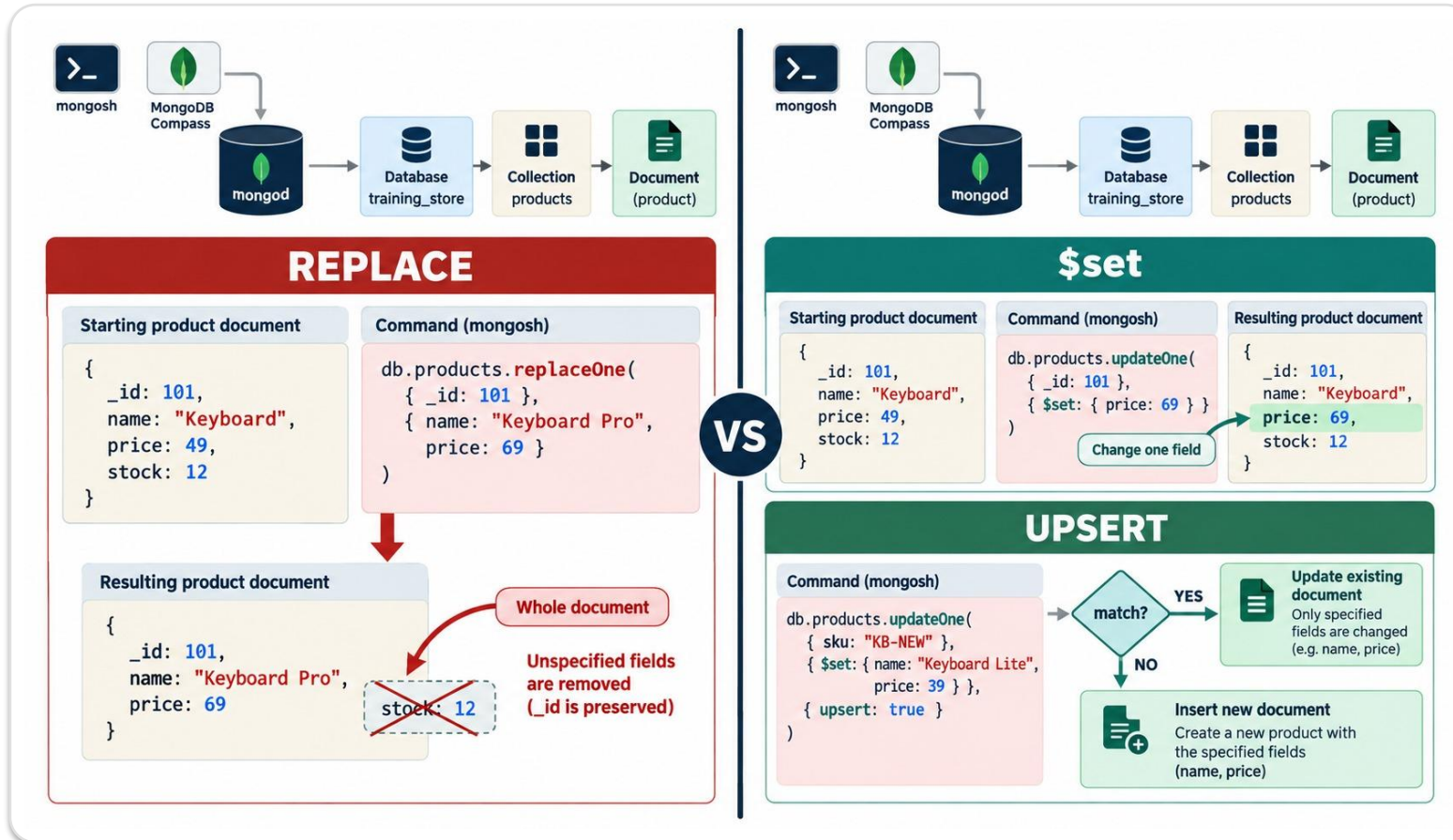
On O5001

```
db.orders.updateOne(
  { orderNumber: "O5001",
    "items.sku": "L100" },
  { $set: {
    "items.$.quantity": 2 } })
// only the L100 line
```

KEY TAKEAWAY Put the array condition in the filter and use \$ for the matched element — or \$[] and arrayFilters for many.

REPLACE VS \$SET

replaceOne() rewrites the document; \$set changes named fields.



replaceOne()

A new body for the whole document; only _id survives.

\$set

Changes the named fields; the rest stays.

Practise on a copy

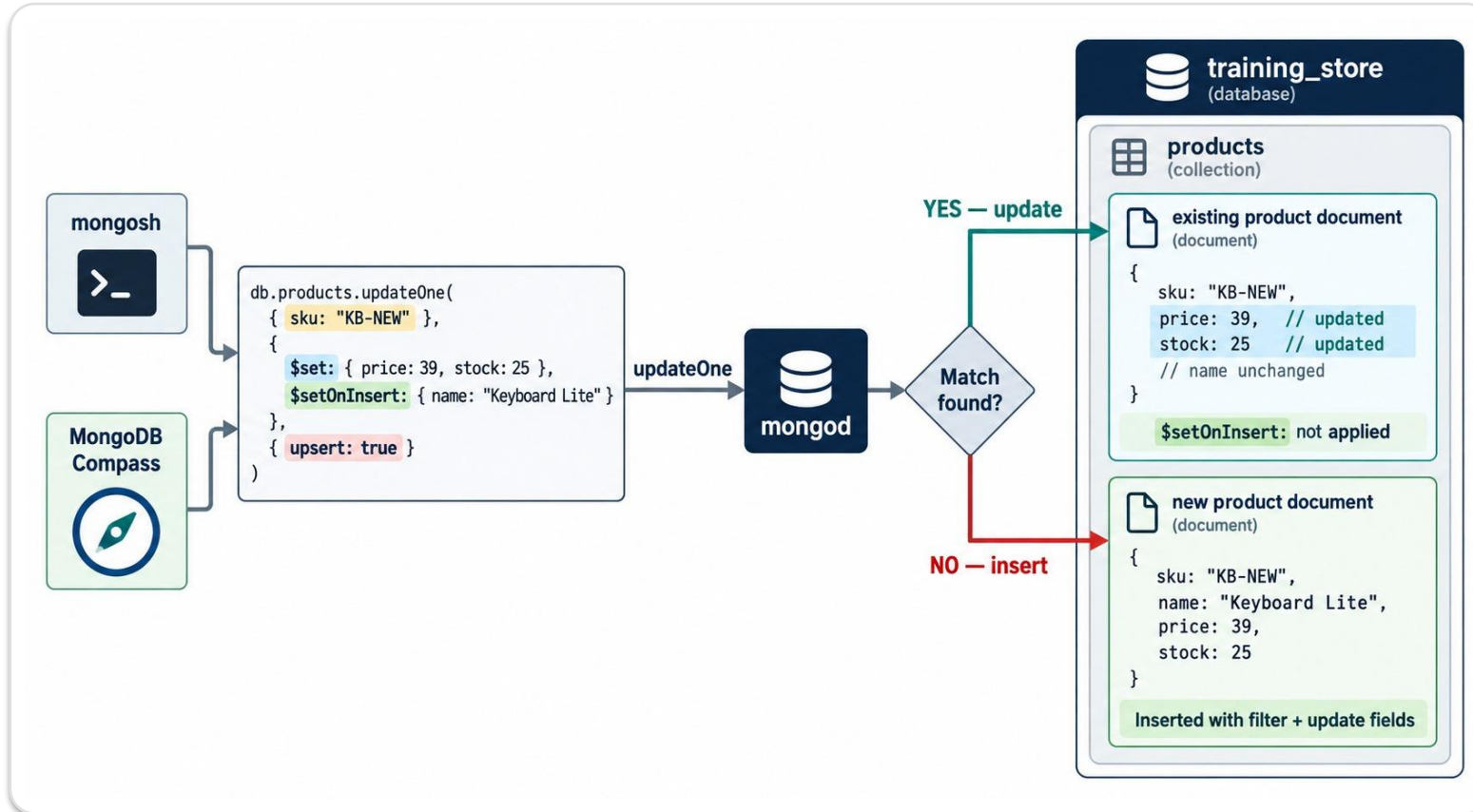
Use a test SKU such as DEMO-REP — never L100.

RULE Replace only when you mean to rewrite every field.

KEY TAKEAWAY replaceOne() rewrites the whole document; when only some fields should change, use update operators.

THE UPSERT PATTERN

One command: update if found, insert if not.



Exercise 4.3 (trimmed)

```
db.products.updateOne(
  { sku: "A700" },
  { $set: {
    name: "Portable Charger",
    updatedAt: new Date() },
    $setOnInsert: {
      createdAt: new Date() } },
  { upsert: true })
```

First run

No match: inserts A700; upsertedCount 1 and the new _id.

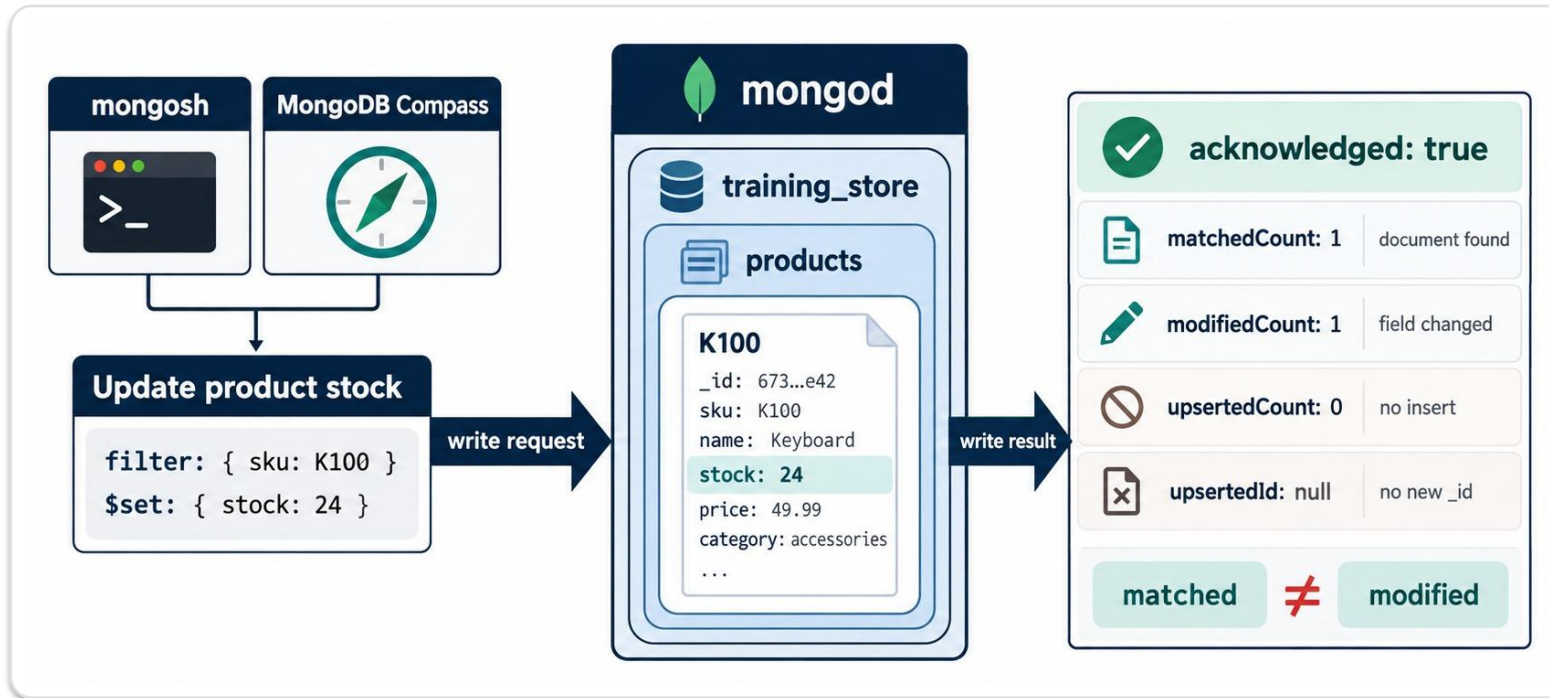
Second run

Matches: updatedAt changes, createdAt stays.

KEY TAKEAWAY An upsert updates when the filter matches and inserts when it doesn't — `$setOnInsert` applies only to the insert.

UNDERSTANDING WRITE RESULTS

Matched is not the same as modified.



acknowledged

The server accepted the write.

matchedCount / modifiedCount

Found / really changed; 1 and 0 means already right.

upsertedId / deletedCount

The new _id of an upsert; documents removed.

ON TRAINING_STORE updateMany({}, { \$pull: { tags: "discontinued" } }) matches 13, modifies 1.

KEY TAKEAWAY Read every field of the write result — matched, modified, upserted and deleted each answer a different question.

PART 5: DELETES, BULK WRITES AND SAFE PRACTICE

Remove data safely, batch writes, and fix queries that go wrong.



SO FAR

- ✓ Insert with real BSON types
- ✓ Update operators change only named fields
- ✓ Upserts insert when nothing matches
- ✓ Write results show what really changed

NEXT QUESTION

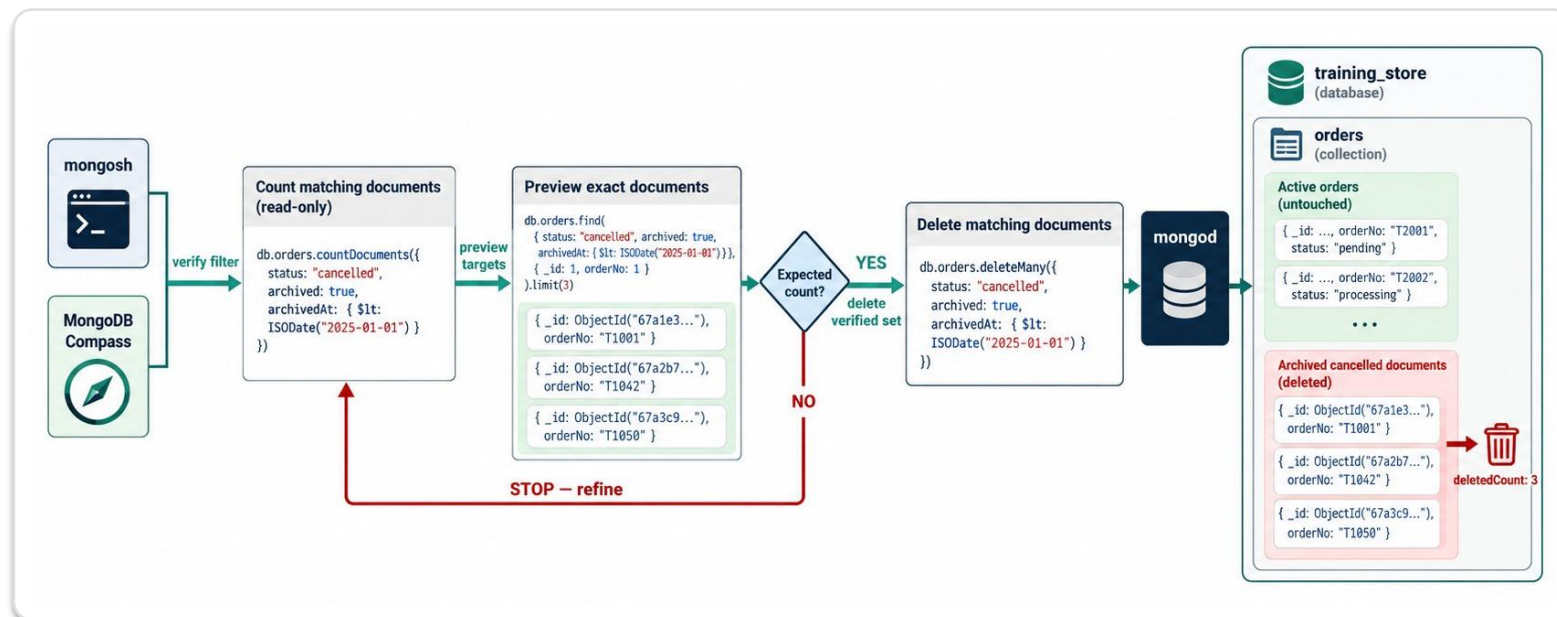
How do we delete and batch writes safely — and debug a query that returns the wrong thing?

IN THIS PART

- 1 deleteOne() and deleteMany()
- 2 Preview, count, then delete
- 3 bulkWrite(): many writes, one result
- 4 Troubleshooting an empty result
- 5 Common mistakes and safe habits
- 6 Module 4 at a glance

DELETING DOCUMENTS SAFELY

Count, preview, check — then delete the verified set.



Demo 4.9

```
const filter =
  { sku: "TEMP-100" }
db.products.find(filter)
db.products
  .countDocuments(filter)
// 1
db.products.deleteOne(filter)
// deletedCount: 1
```

deleteOne / deleteMany

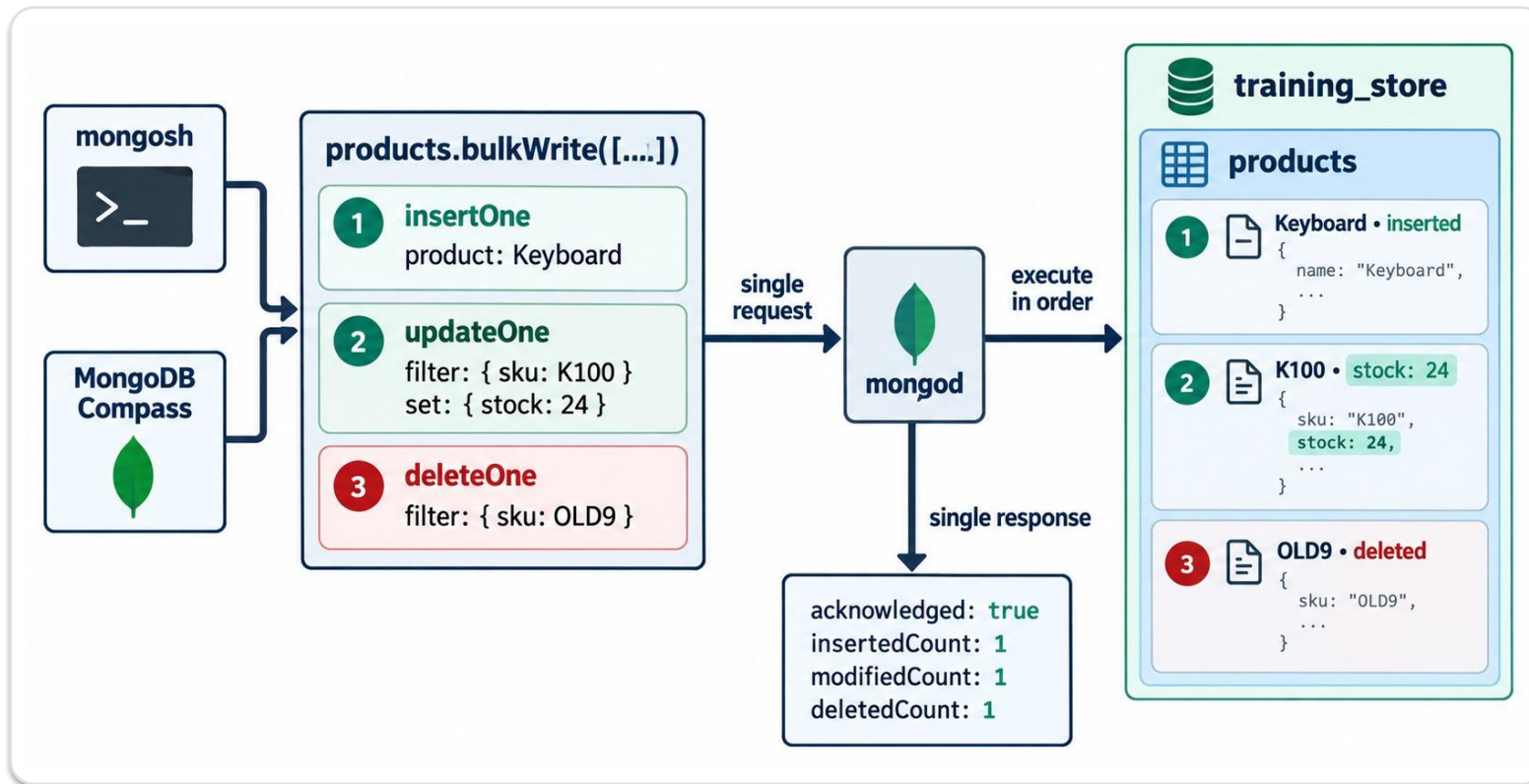
The first match, or every match.

NEVER `deleteMany({})` removes every document.

KEY TAKEAWAY Delete with exactly the filter you previewed and counted — then compare `deletedCount` with that count.

BULK WRITE OPERATIONS

Several writes in one request, one combined result.



Demo 4.10 (trimmed)

```
db.products.bulkWrite([
  { insertOne: { document:
    { sku: "DEMO-BULK" } } },
  { updateOne: {
    filter: { sku: "DEMO-BULK" },
    update: { $set:
      { active: false } } },
  { deleteOne: { filter:
    { sku: "TEMP-100" } } } ])
```

Result

insertedCount 1, matchedCount 1,
modifiedCount 1, deletedCount 0.

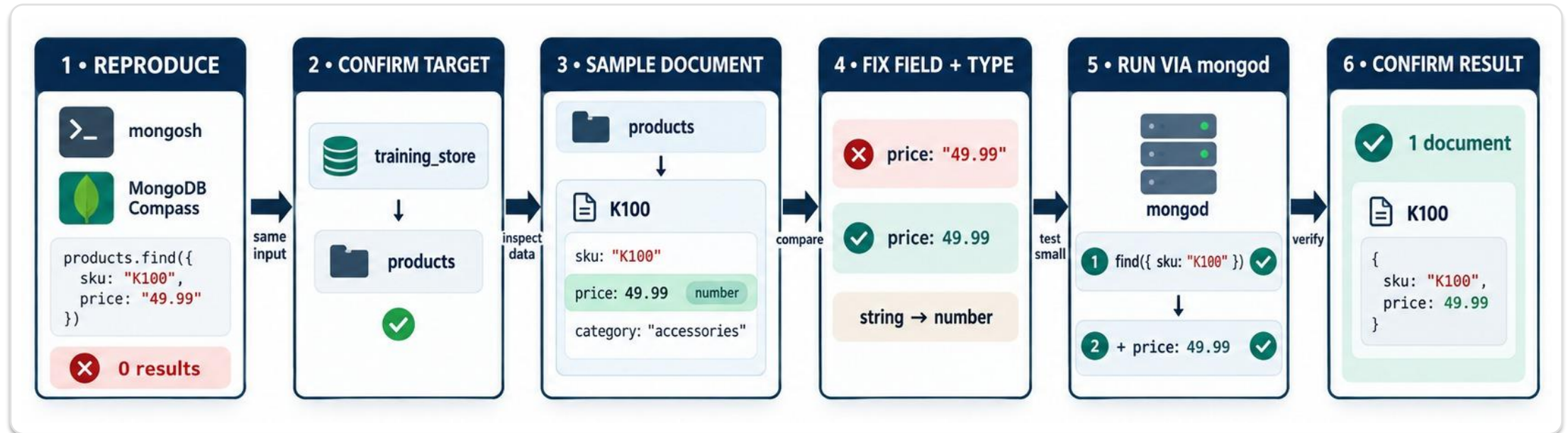
Ordered by default

Stops at the first error; ordered: false carries on.

KEY TAKEAWAY bulkWrite() sends many writes in one round trip — then read each count in the combined result.

QUERY TROUBLESHOOTING WORKFLOW

Reproduce, confirm, sample, fix, test small, confirm.



Reproduce

`find({ price: "49.99" })` returns only XBAD — the broken one.

Compare

`findOne({ sku: "P1001" })` shows a `Decimal128` price.

Fix the type

`{ price: Decimal128("49.99") } → P1001 only.`

RULE Test the smallest query first, then add one condition.

KEY TAKEAWAY When a query returns nothing — or the wrong thing — compare it field by field with a real document.

COMMON QUERY MISTAKES AND SAFE-WRITE HABITS

Seven classic query errors — and the habits that prevent write disasters.

MISCONCEPTION

REALITY

<code>{ Category: "LAPTOP" }</code>	→	Field names are case-sensitive: <code>category</code>
<code>{ price: { \$gt: "100.00" } }</code>	→	Compare money as <code>Decimal128("100.00")</code>
<code>{ attributes.memoryGB: 16 }</code>	→	Quote dotted paths: <code>"attributes.memoryGB"</code>
<code>{ \$gt: { price: ... } }</code>	→	Operator inside the field: <code>{ price: { \$gt: ... } }</code>
<code>updateOne(f, { featured: true })</code>	→	Use an operator: <code>{ \$set: { featured: true } }</code>

ANTI-PATTERNS TO AVOID

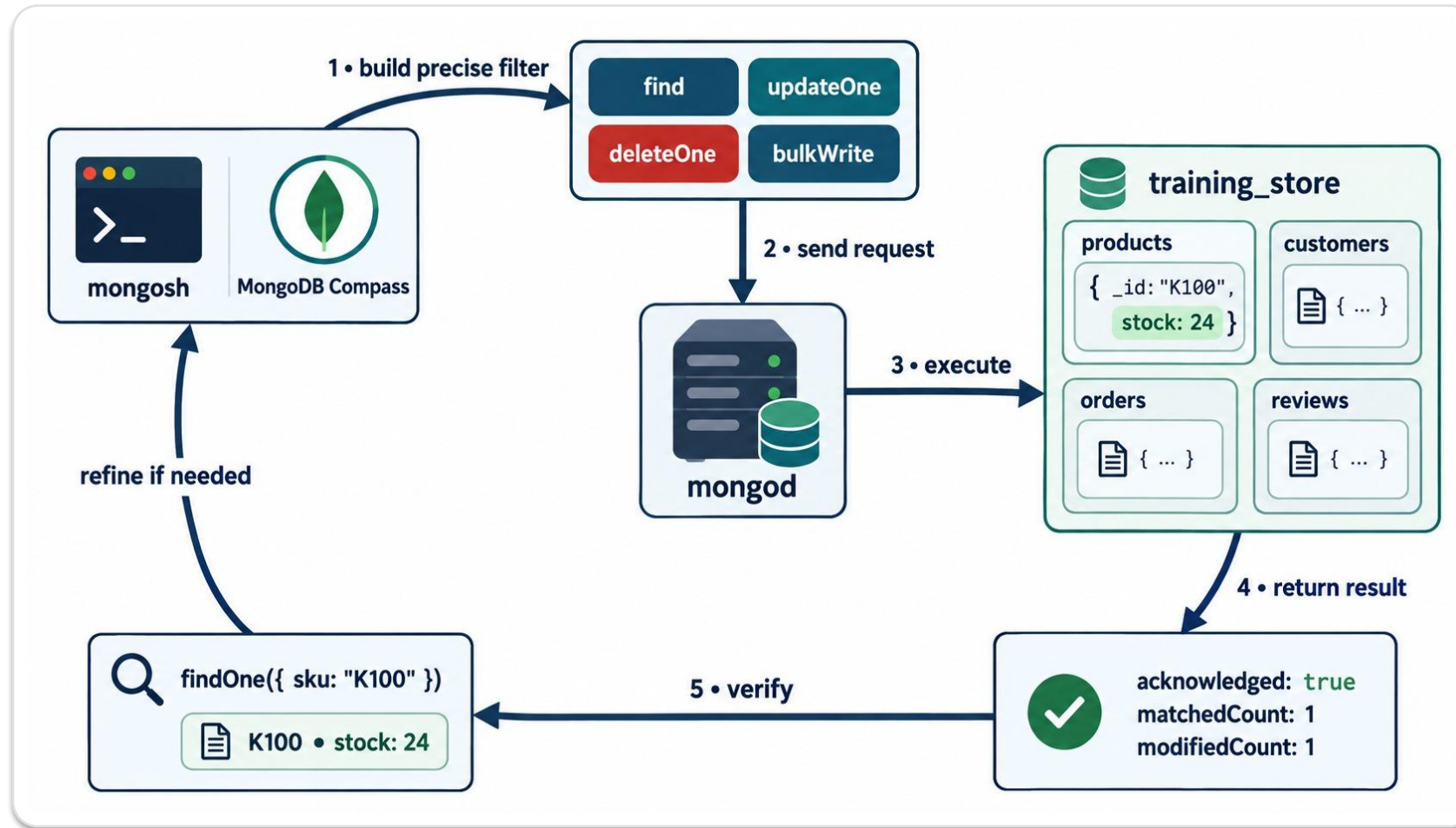
- ✗ `updateMany({})` or `deleteMany({})`
- ✗ `$set` on a parent object instead of one path
- ✗ `$push` where values must be unique
- ✗ `skip()` and `limit()` without `sort()`
- ✗ Writing without reading the result

REMEMBER Preview, count, write, then verify.

KEY TAKEAWAY Most query bugs are a wrong name, case, type or path — most write disasters are an unchecked filter.

MODULE 4 AT A GLANCE

Build a precise filter, send it, execute, read the result, verify.



KEY TAKEAWAY Every query and every write follows one loop: precise filter, execute, read the result, verify.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 findOne() or find() — what does each return?

2 What does the filter {} match?

3 How do you query a nested field?

4 When is \$elemMatch required?

5 What is the projection mixing rule?

6 \$push or \$addToSet — what is the difference?

7 updateOne() with \$set, or replaceOne()?

8 What does upsert: true do?

9 matchedCount 1, modifiedCount 0 — meaning?

10 Why does XBAD miss every price range?

EXERCISE 4.1 — BUILD COMPARISON FILTERS

Module 4 checkpoint A: ranges and lists with Decimal128.

OFFICIAL CHECKPOINT A

15 MIN

labs/day-02/exercises/exercise-4.1-build-comparison-filters.md

SCENARIO

Money in training_store is Decimal128 — except XBAD, planted with a string price.

```
1 products: price > 100
2 products: 50 ≤ price ≤ 500
3 orders:   total > 1000
4 reviews: rating ≥ 4
5 products: not LAPTOP or SHOE
```

YOUR TASKS

- 1 Write 1 and 2 with Decimal128 values
- 2 Write 3 and 4 on orders and reviews
- 3 Write 5 with \$nin
- 4 Explain why XBAD never appears in 1–2

SOLUTION (AFTER DEBRIEF)

- ✓ 1 L100, L110, L190, S200, A500
- ✓ 2 L190, S200, S210, B300, A500
- ✓ 3 Six orders, O5001 to O6401
- ✓ 4 Five of the six reviews
- ✓ 5 Seven products, XBAD included

KEY TAKEAWAY Compare values of the stored BSON type — Decimal128 for money — and check every result against the data.

EXERCISE 4.2 — QUERY ARRAYS OF DOCUMENTS

Module 4 checkpoint B: prove the O5001 \$elemMatch trap.

OFFICIAL CHECKPOINT B

15 MIN

labs/day-02/exercises/exercise-4.2-query-arrays-of-documents.md

SCENARIO

O5001 holds L100 × 1 and A410 × 2. Show why dot notation is not enough.

```
1 "items.sku": "L100"
2 "items.quantity": { $gte: 2 }
3 $elemMatch: L100 and qty ≥ 2
4 1 and 2 with dot notation
5 addresses $elemMatch:
  SHIPPING in Toronto
```

YOUR TASKS

- 1 Run steps 1 and 2; list the orders
- 2 Run steps 3 and 4 side by side
- 3 Name the false positive and why
- 4 Query addresses with \$elemMatch

SOLUTION (AFTER DEBRIEF)

- ✓ 1 O5001, O6101, O6301, O6401
- ✓ 2 Six orders
- ✓ 3 O6401 only
- ✓ 4 O5001 and O6401
- ✓ 5 C101

KEY TAKEAWAY Dot notation lets different elements satisfy different conditions; \$elemMatch makes one element satisfy them all.

EXERCISE 4.3 — PERFORM AN UPSERT

Module 4 checkpoint C: the same upsert, run twice.

OFFICIAL CHECKPOINT C

10 MIN

labs/day-02/exercises/exercise-4.3-perform-an-upsert.md

SCENARIO

A700 is not in the fresh dataset. Run one upsert twice and compare the write results.

```
db.products.updateOne(  
  { sku: "A700" },  
  { $set: { name, category,  
    price, active, updatedAt },  
    $setOnInsert: { createdAt,  
      stockQuantity: 25 } },  
  { upsert: true })
```

YOUR TASKS

- 1 Run the upsert; read the result
- 2 findOne A700; note createdAt
- 3 Run the same upsert again
- 4 Compare both results in two bullets

SOLUTION (AFTER DEBRIEF)

- ✓ Run 1: matched 0, upsertedCount 1
- ✓ Run 2: matched 1, modified 1
- ✓ createdAt unchanged after run 2
- ✓ The filter uses the unique sku

KEY TAKEAWAY Upsert on a unique field and put creation-only fields in \$setOnInsert, so a rerun updates instead of duplicating.

EXERCISE 4.4 — CORRECT UNSAFE OPERATIONS

Module 4 checkpoint D: rewrite three dangerous writes. Don't run them.

OFFICIAL CHECKPOINT D

10 MIN

labs/day-02/exercises/exercise-4.4-correct-unsafe-operations.md

SCENARIO

A teammate wants to run these. Rewrite each one instead.

```
db.products.updateMany({},
  { $set: { active: false } })
db.orders.deleteMany({})
db.customers.updateOne(
  { customerNumber: "C101" },
  { $set: { contact:
    { email: "new@example.com" } } })
```

YOUR TASKS

- 1 Name what each command would damage
- 2 Rewrite 1 with a precise filter
- 3 Rewrite 2 as find → count → delete
- 4 Rewrite 3 with a dotted path

SOLUTION (AFTER DEBRIEF)

- ✓ 1 All 13 products go inactive
- ✓ Safer: { tags: "discontinued" }: 1
- ✓ 2 All 17 orders are deleted
- ✓ 3 C101 loses her phone
- ✓ Use "contact.email" instead

KEY TAKEAWAY Name the blast radius before every write — an empty filter or a replaced parent object is rarely what you mean.

LAB 3 — COMPLEX QUERIES AND UPDATES

Day 2 hands-on: filters, arrays and guarded writes on training_store.

OFFICIAL LAB

75 MIN

labs/day-02/lab3/LAB-3-GUIDE.md

SCENARIO

Reload first. Steps 1–7 read, steps 8–12 write and verify, step 13 is the challenge.

```
# In PowerShell, from the repo root
mongosh "mongodb://localhost:27017" \
  .\datasets\training_store\load.js
```

YOUR TASKS

- 1–4 Filters, ranges and \$nin
- 5–7 Cities, \$elemMatch, PROCESSING
- 8–9 Guarded \$inc, \$addToSet on A410
- 10–12 \$pull, ship O6202, \$rename
- 13 Challenge: one precise filter

EXPECTED RESULTS

- ✓ 1 A410, A400, P1001
- ✓ 6 O5001, O6201, O6305
- ✓ 8 A410 stock 18 → 16
- ✓ 11 Second ship matches 0
- ✓ 13 A410, P1001, B310, B300

KEY TAKEAWAY Lab 3 puts the whole module together: precise filters first, then writes guarded by the filter and verified by the result.

MODULE 4 SUMMARY AND WHAT'S NEXT

From precise filters to safe, verified writes.

YOU CAN NOW

- ✓ Read with `find()`, `findOne()` and precise filters
- ✓ Project, sort, page, count and list distinct values
- ✓ Combine comparison, logical and element operators
- ✓ Query nested fields and arrays with `$elemMatch`
- ✓ Insert, update, upsert and replace documents
- ✓ Delete safely and read every write result

NEXT: MODULE 5 — THE AGGREGATION FRAMEWORK

- 1 How does a filter become a `$match` stage?
- 2 How does `$group` total the orders?
- 3 How do stages chain into a pipeline?

BRING ALONG Your Lab 3 queries — and reload `load.js` so the counts start fresh.

KEY TAKEAWAY Module 5 reuses today's filters and projections as the first stages of an aggregation pipeline.